THOMPSON RIVERS UNIVERSITY

A Hybrid Optimization Framework for Efficient Feature Selection in Credit Scoring

By

Sagar

A PROJECT REPORT IS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

Master of Science in Data Science

KAMLOOPS, BRITISH COLUMBIA

April, 2025

SUPERVISOR

Dr. Mohamed Tawhid

ABSTRACT

Feature selection (FS) is a critical step in high-dimensional machine learning tasks, particularly in credit scoring, where classification accuracy and model interpretability are essential. As FS is an NP-hard problem, metaheuristic algorithms are commonly used to find near-optimal subsets of features efficiently. This study evaluates thirteen nature-inspired metaheuristic algorithms on seven publicly available credit scoring datasets using Logistic Regression, Naïve Bayes, and Random Forest classifiers. The Friedman test at a 5% significance level is applied to rank the algorithms based on classification accuracy metrics including mean, best, worst, and standard deviation. Based on the top-ranked methods, a new hybrid algorithm, MSO-GOA, is proposed by integrating Mirage Search Optimization (MSO) and Golf Optimization Algorithm (GOA) to balance exploration and exploitation. MSO-GOA is implemented within a wrapper-based feature selection framework. Its performance is compared with its individual components and five existing algorithms from the literature using the overall mean accuracy across all datasets. Experimental results show that MSO-GOA consistently selects compact feature subsets and outperforms other methods in terms of classification accuracy and robustness, demonstrating its suitability for credit scoring applications.

Key Words: Feature selection, metaheuristic optimization, hybrid algorithm, wrapper approach, credit scoring, classification.

ACKNOWLEDGEMENTS

I respectfully acknowledge that I live, learn, and conduct my research on the traditional and unceded territory of the Tk'emlúps te Secwépemc, within Secwépemc'ulucw, the traditional lands of the Secwépemc people. I would like to sincerely thank my supervisor, Dr. Mohamed Tawhid, for his constant guidance, and committee member, Dr. Mirduala Sharma for encouragement, and support throughout this research project. Their insights and mentorship have been invaluable to my learning and growth. I am also grateful to the faculty and staff of Thompson Rivers University for providing a supportive academic environment. Finally, I express my heartfelt thanks to my family and friends for their endless encouragement and patience during this journey.

Chapter 1

Introduction

1.1 Feature Selection and the Need for Optimization

Feature selection (FS) is a fundamental step in machine learning, particularly when dealing with high-dimensional datasets where irrelevant, redundant, or noisy features can degrade the performance of predictive models[4] [1]. By identifying and selecting a subset of the most informative features, FS not only improves classification accuracy and generalization but also reduces computational costs, enhances model interpretability, and mitigates the risk of overfitting[1], [2]. The importance of FS becomes even more pronounced in sensitive domains such as finance and healthcare, where transparent, accurate, and efficient decision-making systems are required hyperlink nssibi[6].

Formally, the FS problem is a combinatorial optimization challenge. Given a dataset with n features, the total number of possible feature subsets

is 2^n, making exhaustive search computationally infeasible for even moderately sized feature spaces. This complexity renders FS an NP-hard problem. Traditional FS techniques such as filter, wrapper, and embedded methods provide viable approaches for specific scenarios, but they often struggle when applied to large-scale or highly nonlinear feature spaces[7]. This limitation has led to growing interest in nature-inspired metaheuristic algorithms for solving FS problems efficiently[3]].

Metaheuristics are high-level, stochastic optimization strategies inspired by natural phenomena such as evolution, swarm behavior, and physical processes[1], [3]. Their strength lies in balancing exploration (global search) and exploitation (local refinement), allowing them to navigate large, non-convex, and multimodal search spaces without the need for gradient information or problem-specific assumptions. Over the past decade, numerous metaheuristic algorithms—such as Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), Grey Wolf Optimizer (GWO), Whale Optimization Algorithm (WOA), and others—have been adapted for FS tasks using binary encoding schemes[3].

1.2 Metaheuristic Algorithms for Feature Selection

The adaptation of metaheuristics for FS typically involves binary conversion mechanisms and integration within a wrapper-based framework, wherein a classifier evaluates the predictive performance of each candidate feature subset [2]. Compared to filter-based methods, which rely on statistical metrics,

wrapper-based FS evaluates feature subsets using actual model performance, thus capturing nonlinear dependencies and interaction effects among features. Although wrapper methods are computationally more intensive, they generally yield better classification accuracy and are well-suited to metaheuristic search[2].

Recent studies have demonstrated that different metaheuristic algorithms exhibit varying strengths in global exploration, convergence speed, and solution diversity. However, no single algorithm consistently outperforms others across all datasets and problem types—a consequence of the No Free Lunch (NFL) theorem. As a result, there is growing interest in hybrid metaheuristic algorithms that combine the complementary advantages of two or more existing strategies. Such hybrids aim to enhance the search process by fusing global and local behaviors in a synergistic manner [4], [5].

1.3 Credit Scoring as a Case for Feature Selection

Credit scoring is a critical application area for classification models, wherein the objective is to categorize borrowers as either "good" (likely to repay) or "bad" (likely to default). This classification task directly influences credit approval, interest rate determination, and risk management decisions in financial institutions. Given the high economic and regulatory stakes, credit scoring systems must be both accurate and interpretable.

Modern credit datasets are increasingly complex, integrating demographic, transactional, behavioral, and alternative financial data. The high dimen-

sionality of such data makes FS essential to reduce model complexity, enhance predictive performance, and improve interoperability [24]. Moreover, regulators such as the Basel Committee and frameworks like GDPR demand that automated decision-making systems be explainable and fair, further reinforcing the importance of selecting relevant and non-discriminatory features.

In this study, FS is approached as a discrete optimization problem within the credit scoring domain. Seven publicly available credit scoring datasets—varying in size, dimensionality, and class distribution—are employed to ensure the robustness and generalizability of results. Wrapper-based FS is performed using three widely-used classifiers: Logistic Regression, Naïve Bayes, and Random Forest[2]. This multi-classifier evaluation allows for a comprehensive assessment of feature subset quality.

1.4 Research Contribution and Hybrid Algorithm Motivation

This research addresses two major objectives: (i) benchmarking the performance of existing metaheuristic algorithms for feature selection in credit scoring, and (ii) developing a hybrid metaheuristic that outperforms individual algorithms in accuracy, stability, and feature compactness.

To this end, thirteen recent nature-inspired metaheuristic algorithms were selected based on their diversity in inspiration and search behavior. These include algorithms such as Mirage Search Optimization (MSO)[11], Golf Optimization Algorithm (GOA)[13], Beluga Whale Optimization (BWO)[17], and others. Each algorithm was adapted to a binary setting for FS and eval-

uated across all seven datasets and three classifiers using a wrapper-based evaluation.

To statistically assess and rank algorithm performance, the Friedman test—a nonparametric statistical test suitable for comparing multiple algorithms over multiple datasets—was applied at a 5% significance level. Based on this analysis, top-performing algorithms were identified and used as the foundation for designing a hybrid optimization strategy.

A new hybrid metaheuristic algorithm, termed **MSO-GOA**, is proposed in this study. MSO-GOA integrates the exploration strength of MSO with the exploitation refinement of GOA. MSO contributes long-range, mirage-inspired global jumps, while GOA offers precision control through golf swing dynamics, including adaptive distance reduction over iterations. The combination is structured to enhance convergence reliability, search diversity, and robustness to local minima.

1.5 Experimental Framework and Evaluation Strategy

The proposed MSO-GOA algorithm was implemented within a wrapper-based FS framework, where each feature subset was evaluated using the chosen classifier's accuracy[2]. The performance of MSO-GOA was compared to its individual components (MSO and GOA) and five well-established metaheuristic algorithms from the literature. The evaluation was based on classification accuracy, the size of the selected feature subset, and the stability of results across multiple runs.

To summarize algorithm performance across datasets, the overall mean accuracy was computed, and comparative rankings were established. The inclusion of the Friedman test ensured that observed performance differences were statistically valid and not due to random chance. This dual evaluation strategy—quantitative comparison and statistical validation—provides strong empirical support for the effectiveness of the proposed hybrid.

1.6 Key Contributions

This study offers the following contributions to the field of feature selection and optimization:

- A comprehensive performance benchmarking of thirteen recent metaheuristic algorithms for feature selection using seven real-world credit scoring datasets.
- Statistical ranking of algorithms using the Friedman test at a 5% significance level, identifying top performers for hybridization.
- Development of a novel hybrid algorithm, MSO-GOA, that combines global and local search strengths to improve feature selection performance.
- Comparative evaluation of the hybrid algorithm against its individual base algorithms and five literature-based methods in terms of classification accuracy, robustness, and subset compactness.
- Insights into algorithm–classifier compatibility and practical implications for interpretable credit scoring models.

Chapter 2

Literature Review

2.1 Introduction to Feature Selection

Feature selection is a critical preprocessing step in machine learning, particularly in high-dimensional datasets where the presence of redundant, irrelevant, or noisy features can significantly degrade model performance [1], [3]. The primary objective of feature selection is to identify a minimal subset of features that maintains or improves the performance of a learning algorithm [4], [2]. This not only reduces training time and computational overhead but also enhances generalization by mitigating overfitting and increasing model interpretability [1], [2], [5].

Feature selection techniques are generally categorized into three types: filter, wrapper, and embedded methods [2], [6]. Filter methods rank features based on statistical measures such as mutual information, Pearson correlation, or chi-square tests. Wrapper methods, on the other hand, evaluate

subsets of features by training and testing a model using a specific learning algorithm, making them computationally expensive but often more accurate [3], [2]. Embedded methods perform feature seleciton.

2.2 Traditional Feature Selection Techniques

Conventional feature selection techniques have long served as the foundation of data dimensionality reduction. Techniques such as Information Gain, Gain Ratio, ReliefF, and Fisher Score have been widely employed in filter-based approaches due to their speed and ease of implementation. While effective in some contexts, they typically evaluate each feature independently and thus may overlook critical interactions among features.

Wrapper methods, such as Sequential Forward Selection (SFS) and Recursive Feature Elimination (RFE), offer improved accuracy by considering feature subsets as a whole. However, their major drawback lies in the combinatorial explosion of possible feature subsets, especially when applied to large-scale datasets. As such, they can become prohibitively expensive computationally. Embedded methods, like L1-regularized logistic regression (Lasso) or tree-based feature importances in Random Forests, partially overcome these limitations by integrating feature selection into model construction. Still, their performance is closely tied to the specific algorithm used, and they lack general applicability across diverse model architectures.

Given these limitations, researchers have increasingly turned to metaheuristic optimization techniques to address the challenges of scalability, global search, and interaction awareness in feature selection tasks.

2.3 Metaheuristic Algorithms in Feature Selection

Metaheuristic algorithms are general-purpose optimization strategies inspired by biological, social, or physical systems [1], [4]. These algorithms are particularly effective for solving high-dimensional, nonlinear, and combinatorial problems, making them suitable for feature selection tasks [2], [5]. Unlike traditional deterministic techniques, metaheuristics do not require gradient information or domain-specific assumptions, allowing them to flexibly explore the solution space and escape local optima [6].

Feature selection is a combinatorial problem where the objective is to choose an optimal subset of features from a given set, often represented as binary vectors. Each feature can be either selected (1) or excluded (0), resulting in a search space of size 2^n, where n is the number of features [1], [3]. Exhaustive search in such spaces is computationally infeasible, which is why metaheuristic approaches are widely adopted [2], [4]. These algorithms balance exploration (global search) and exploitation (local refinement) to effectively navigate the solution landscape [6], [5].

Metaheuristics have shown considerable promise in selecting feature subsets that yield high classification accuracy, minimal redundancy, and low model complexity [1], [2]. Their ability to capture non-linear interactions among features enables them to outperform traditional filter and wrapper methods, especially in domains with complex relationships and noisy data [3], [5]. Various population-based algorithms have been adapted for feature selection, often through binarization techniques, and evaluated using

standard metrics such as accuracy, number of features selected, and computational time [2], [6].

Additionally, metaheuristic algorithms can be easily adapted to different classifiers in a wrapper-based framework. Their flexibility supports generalization across multiple classification models, making them attractive for real-world applications where robustness is a key requirement [4], [6]. The consistent performance of these algorithms across a range of datasets and classifiers has motivated ongoing research into their hybridization, parameter tuning, and comparative benchmarking [5], [2].

2.4 Metaheuristic Algorithms for Feature Selection

Over the past years, numerous new metaheuristic algorithms have been proposed and applied to feature selection. Many of these recent algorithms have bio-inspired or physics-inspired origins. We briefly review thirteen notable algorithms by describing their inspiration, main operators, and binary adaptation techniques. Relevant citations for each are provided from existing literature.

2.4.1 Mirage Search Optimization (MSO)

The Mirage Search Optimization (MSO) algorithm (proposed in 2025 by He et al) [11]. is inspired by the mirage phenomenon in deserts. A mirage occurs due to refracted light, creating illusions of water or distant objects. MSO

models two mirage-inspired strategies: a superior mirage phase and an inferior mirage phase. The superior mirage strategy simulates the scenario where an inverted image appears above the true object (analogous to exploration far from the current position), while the inferior mirage strategy simulates a normal image below the horizon (analogous to exploitation near the ground truth). These two update operators respectively drive global exploration and local exploitation in the search space. Additional mechanisms like Fitness-Distance Balance (FDB) and Fitness-Distance Constraint (FDC) have been incorporated into variants of MSO to enhance convergence. MSO has shown competitive performance on standard benchmarks and engineering problems, e.g. path planning, outperforming many classical optimizers.

2.4.2 Preschool Education Optimization Algorithm (PEOA)

The Preschool Education Optimization Algorithm (PEOA) is a human behavior-inspired metaheuristic introduced in 2023 by Trojovský [12]. It models the learning stages of preschool children under guidance of a teacher. Specifically, PEOA's search process is divided into three phases: (i) Gradual growth of the teacher's influence – simulating how a teacher increasingly shapes the learning environment, (ii) Individual knowledge development guided by the teacher – simulating students learning and improving with teacher feedback, and (iii) Individual increase of self-knowledge and awareness – simulating the child's self-driven learning and discovery. Each phase corresponds to a set of operators that update candidate solutions. For example, early iterations emphasize "teacher influence" (exploration by guiding solutions towards a mentor position), whereas later iterations emphasize "self-awareness" (exploitation by fine-tuning solutions independently). PEOA was validated on

numerous benchmarks (52 standard test functions and the CEC2017 suite) and showed a balanced exploration–exploitation capability. Statistically, it outperformed several well-known algorithms in those tests, establishing itself as a potent new optimizer.

2.4.3 Golf Optimization Algorithm (GOA)

The Golf Optimization Algorithm (GOA) is a 2023 game-based metaheuristic inspired by the sport of golf [13]. GOA imitates the behavior of golfers attempting to get a ball into the hole in as few strokes as possible. The algorithm is structured in two main phases reflecting the gameplay: an exploration phase analogous to a long-distance drive (tee shot) and an exploitation phase analogous to precision putting on the green. In the exploration phase (Phase 1), each candidate solution ("ball") is struck with a powerful shot to move it closer to the target (hole) – mathematically, this is modeled by updating the solution position using a random long-distance move influenced by the best solution (simulating a strong swing). In the exploitation phase (Phase 2), the focus is on fine adjustments: solutions make smaller, precise moves ("low-power shots") to finally sink into the target. GOA's update equations for these phases were derived to balance large exploratory jumps and small convergent steps. By alternating these two phases each iteration, GOA gradually guides solutions from a broad search towards the optimum. Montazeri et al. demonstrated GOA's effectiveness on benchmark functions and a resilient energy commitment problem, where GOA obtained competitive results and showed robust convergence behavior

2.4.4 Blood-Sucking Leech Optimization (BSLO)

Blood-Sucking Leech Optimizer (BSLO) is a nature-inspired algorithm introduced by Bai et al. in 2024 [14], motivated by the foraging behavior of leeches in swampy rice fields. Leeches exhibit a mix of movement strategies when seeking hosts for blood meals. BSLO simulates both directional and non-directional search strategies, as well as the leech's ability to switch tactics and retrace its path. In the algorithm, a directional search might represent a leech sensing a gradient (e.g. heat or vibration from a potential host) and moving towards it (exploitation), whereas a non-directional search represents a leech casting around randomly when no cue is detected (exploration). The switching mechanism allows BSLO to alternate between these modes based on the situation (akin to a leech alternating between active pursuit and random swimming). Additionally, a retracing strategy is modeled, which reflects how a leech might backtrack if it overshoots or loses the trail – in optimization terms, this helps the algorithm refine solutions around promising regions it may have passed. Through these behaviors, BSLO manages exploration (random wandering in the "water") and exploitation (targeted approach when a host is near). The original BSLO paper validated the algorithm on standard benchmarks and demonstrated its competitive performance versus other recent metaheuristics. For example, in engineering test problems, BSLO achieved high-quality solutions by effectively balancing its search modes.

2.4.5 Giant Armadillo Optimization (GAO)

Giant Armadillo Optimization (GAO) is a bio-inspired metaheuristic based on the foraging and hunting behavior of giant armadillos (Priodontes maximus), introduced in 2023 by Alsayyed et al [15]. The giant armadillo's main prey are termites and ants, which it obtains by locating termite mounds and digging them open. GAO simulates this with two primary phases: exploration = moving towards termite mounds and exploitation = digging into the mound. During exploration, candidate solutions wander the search space analogous to an armadillo sniffing out or moving toward potential termite mounds (this is modeled via random walks biased by global best solutions, to find promising regions). In the exploitation phase, the algorithm intensifies the search around a detected "mound" – analogous to the armadillo digging vigorously at a specific location to retrieve food. Mathematically, GAO uses the positions of the current best solutions to guide other agents, and updates each agent's position by a combination of subtractive movement (simulating digging in or exploitation) and exploratory jumps towards new targets. The original GAO study confirmed its effectiveness on standard test suites (CEC2017, etc.) and noted its novelty: few prior algorithms had drawn inspiration from armadillo behavior. Because armadillos intelligently alternate between relocating to new mounds and exploiting known ones, GAO achieved a good exploration–exploitation trade-off.

2.4.6 Subtraction-Average-Based Optimizer (SABO)

The Subtraction-Average-Based Optimizer (SABO) is an evolutionary algorithm inspired by mathematical concepts, proposed by Trojovský and De-

hghani in 2023 [16]. Unlike many bio-inspired algorithms, SABO does not stem from an animal or physical phenomenon, but from a numeric operation metaphor. The fundamental inspiration of SABO is to use the differences and averages of candidate solutions to guide the search. In each iteration, SABO computes certain averages of the positions of all search agents and uses the subtraction (difference) between an agent's position and this average to update the agent. Intuitively, an agent moves in the direction of the average position of the swarm or away from it, depending on whether moving closer improves fitness or not. Additionally, SABO considers the sign of the objective value differences between pairs of solutions as part of its update rule. This strategy enables a balance: if an agent is worse than the average, it moves towards the better region (improving convergence), whereas if it's better, it explores by moving away to find potentially better solutions. The algorithm's operators include calculating the global mean position and then adjusting each individual by a fraction of the difference (current – mean) or (mean – current) to explore new points. The original SABO paper demonstrated its efficacy on various optimization problems, often obtaining superior results by virtue of this arithmetic mean guidance. In summary, SABO's metaphor is purely mathematical – leveraging arithmetic means and differences – but it effectively mimics social learning where individuals gravitate around a group norm or diverge from it to search further.

2.4.7 Beluga Whale Optimization (BWO)

Beluga Whale Optimization (BWO) is a swarm algorithm inspired by beluga whale behaviors, first introduced by Zhong et al. in 2022 [17]. It is one of the marine predator-inspired algorithms, specifically modeling unique aspects of

beluga whale social hunting. BWO defines three phases in its process, analogous to belugas' activities: (1) Pair Swimming – representing exploration, where whales move in pairs and cover large distances in search of prey, (2) Prey (Foraging) – representing exploitation, where whales focus on a detected prey, and (3) Whale Fall – a phase representing a sudden or drastic move, conceptually inspired by the ecological phenomenon of a "whale fall" (when a whale carcass falls to the ocean floor). In the algorithm, the whale fall phase is treated as a diversification strategy that can reposition a solution far in the search space (preventing stagnation). BWO uses self-adaptive parameters to regulate the transition between exploration and exploitation. For example, it employs a balance factor and a probability for triggering the whale fall event, which decrease over iterations to shift the algorithm from exploration to exploitation smoothly. A Lévy flight random walk is also incorporated during exploitation to mimic the belugas' irregular movement and improve global search capability. This combination of social swimming patterns and random dives enabled BWO to perform competitively on numerous benchmarks. Experiments have shown BWO achieving top ranks on complex functions and proving effective on engineering problems, thanks to its mix of smooth convergence (via pair-swim and prey phases) and occasional jumps (via whale fall).

2.4.8 Dandelion Optimization Algorithm (DOA)

The Dandelion Optimization Algorithm (DOA) is a nature-inspired method based on the seed dispersion process of dandelion plants. Proposed in 2021–2022 by Liu et al. [18], DOA captures how dandelion seeds ride the wind to find fertile ground. The algorithm's mathematical model consists of four stages:

Initialization, Ascent, Descent, and Landing.

These stages mirror the journey of a dandelion seed: initially, seeds are generated (random initialization of candidate solutions), then they ascend into the air carried by wind, eventually descend, and finally land on the ground.

In DOA's ascent phase, the seeds (solutions) are carried upward and outward — this is modeled by diversifying the population. The ascent can be influenced by factors like wind speed and weather. For example, under "sunny" conditions with strong wind, seeds disperse farther. DOA simulates this behavior using a log-normal distributed random step that allows solutions to move farther from their origin.
Under "rainy" or mild wind conditions, dispersion is shorter. This weather-based modulation adds a random yet controlled exploration mechanism: it uses a log-normal factor to allow occasional long flights (exploration boost).

In the descent phase, seeds start to fall — DOA begins to reduce the randomness of movement (step sizes shrink adaptively) so that solutions converge. An adaptive parameter α gradually decreases over iterations to limit how far a seed can move as time goes on.

Finally, in the landing phase, seeds settle — DOA likely evaluates where seeds landed and retains the best candidates. Essentially, the algorithm combines wide initial dispersion with gradually intensifying search near promising areas, analogous to seeds first scattering widely, then many landing near suitable spots.

This approach was successfully applied to antenna array design, where

DOA optimized beam patterns effectively. Overall, DOA's inspiration is rooted in the natural spreading strategy of dandelions, enabling both randomness (to explore new "ground") and convergence (as seeds accumulate in fertile zones).

2.4.9 Marine Predators Algorithm (MPA)

The Marine Predators Algorithm (MPA) is a 2020 nature-inspired metaheuristic by Faramarzi et al [19]. that emulates the optimal foraging strategies of marine predators in the ocean. MPA is grounded in predator–prey dynamics, particularly the encounter rate policy – the idea that predators and prey move in ways that affect how often they encounter each other. The algorithm mixes Lévy flights and Brownian motion to simulate different movement patterns of marine hunters. In MPA, during the early iterations (when exploration is key), prey are considered faster than predators, leading predators (candidate solutions) to perform Lévy flight moves (long, sporadic jumps) to search broadly. As iterations progress, the roles switch – predators become faster relative to prey, so they switch to Brownian motion (short-range random walk) to fine-tune around the prey's position (exploitation). MPA effectively has three stages corresponding to different predation behaviors: high exploration (Lévy-based), balanced phase, and high exploitation (Brownian-based), each lasting a third of the iterations as originally defined. It also incorporates concepts like "encircling prey" and "attacking prey" analogous to how real marine predators circle shoals of fish and then strike. Empirical results showed MPA to be very powerful: it ranked among the top optimizers on the CEC2017 competition problems, even outperforming or matching state-of-the-art algorithms like CMA-ES in many cases. The

authors highlighted MPA's balance of exploration/exploitation and its stability, crediting the optimal foraging theory for these strengths.

2.4.10 Whale Optimization Algorithm (WOA)

The Whale Optimization Algorithm (WOA) is a well-known bio-inspired metaheuristic introduced by Mirjalili and Lewis in 2016 [20]. WOA is inspired specifically by the humpback whale's hunting behavior – notably the bubble-net feeding strategy. Humpback whales often encircle schools of fish and create a spiral of bubbles to herd and trap them near the surface. WOA abstracts this into two main operators: encircling prey and bubble-net attacking (spiral updating). Additionally, a random exploration operator represents searching for prey when a whale hasn't found a target. In the algorithm, encircling is modeled by moving each candidate solution towards the current best solution (the whale recognizes prey and circles closer). This is implemented via coefficient vectors $\vec{A}, \vec{C}$ that dynamically shrink, reducing the distance to the best solution over time. The bubble-net attack is modeled in two ways: a "shrink circle" mechanism (linearly decreasing $\vec{A}$ to encourage exploitation) and a "spiral" mechanism (each whale updates its position along a spiral path towards the best solution). With probability p, WOA chooses between these two during iterations. If $p < 0.5$, it performs the encircling (with either a spiral or linear reduction); if $p \geq 0.5$, it performs a random search move (the whale searches for new prey). This random search is typically a random position outside the bubble net, ensuring exploration. The elegant bubble-net spiral equation in WOA enables exploitation around the best candidate, while the random search maintains diversity. WOA gained rapid popularity due to its simplicity and performance, and it has since been

applied to many engineering and machine learning problems. A systematic review noted WOA's wide adoption and effectiveness across domains.

2.4.11 Sine Cosine Algorithm (SCA)

Mirjalili [21] introduced the Sine Cosine Algorithm, a population-based metaheuristic that iteratively updates candidate solutions using trigonometric sine and cosine functions. At each iteration, SCA moves solutions either towards or away from the current best solution (called the destination point) by adding a randomized sine or cosine term to each position. This mechanism enables a smooth transition from exploration to exploitation of the search space, as the influence of the sine/cosine perturbation is adaptively reduced over time. The original study demonstrated that SCA converges rapidly and performs competitively on a suite of benchmark optimization problems, matching or outperforming other contemporary algorithms

2.4.12 Moth Swarm Algorithm (MSA)

Mohamed et al. [22] developed the Moth Swarm Algorithm, inspired by the navigation behavior of moths orienting themselves by moonlight (celestial navigation). The MSA employs a hybrid search strategy with two complementary operators: a diversity-guided crossover combined with Lévy-flight mutation to broadly explore the solution space (analogous to a reconnaissance phase), and an associative learning mechanism with immediate memory to intensify the search around promising regions (a celestial navigation phase). Additional techniques, such as adaptive Gaussian walks and spiral motions,

are incorporated to mimic natural moth movements and enhance the balance between exploration and exploitation. In its original presentation, MSA was applied to complex optimal power flow optimization cases, where it achieved superior results under various scenarios compared to other heuristic optimizers.

2.4.13 Salp Swarm Algorithm (SSA)

Mirjalili et al. [23] introduced the Salp Swarm Algorithm (SSA) as a novel swarm-based metaheuristic inspired by the collective behavior of salpidae (marine salps) forming chain-like swarms in the ocean. The authors were motivated by the absence of any salp-inspired optimizer in the literature and the No-Free-Lunch principle suggesting new algorithms may yield better results on certain problems. Biologically, salps move by pumping water through their barrel-shaped, jellyfish-like bodies and often link into long chains for more efficient locomotion and foraging. SSA mathematically models this behavior by dividing the population into a leader (the first salp in the chain) and followers. The leader's position is updated toward a food source (analogous to the best solution found so far) using a stochastic step influenced by three random coefficients. In particular, the leader moves toward the target in each dimension either forward or backward depending on a random sign (coefficient c_3), with a random step size c_2, while an exponential coefficient $c_1 = 2\exp[-(4l/L)^2]$ decreases over the course of iterations to balance exploration and exploitation. Each follower in the chain updates its position based on a simplified Newtonian motion formula, effectively moving to the midpoint between its current position and the position of the salp directly ahead of it in the chain. This leads to a coordinated "swarming" movement

where follower salps gradually converge toward the leader's path. If any salp goes beyond the search space bounds, its position is set to the boundary to keep solutions feasible. Notably, SSA also retains the best solution found (the food source position) at each iteration, ensuring the swarm does not lose track of the global best even if individuals move suboptimally.

For performance evaluation, the original study tested SSA on a suite of 19 standard benchmark functions categorized as unimodal, multimodal, and composite functions. This diverse test set challenges an optimizer's ability to intensify search on unimodal landscapes (with a single global optimum) and to diversify on multimodal and composite functions, which contain numerous local optima or combine multiple landscape features. SSA was compared against several well-known algorithms, including Particle Swarm Optimization (PSO), Genetic Algorithm (GA), Gravitational Search Algorithm (GSA), Bat Algorithm (BA), Flower Pollination Algorithm (FPA), Firefly Algorithm (FA), and others, under identical population size and iteration settings. All algorithms were run for multiple independent trials (30 runs each), and performance was measured in terms of the best objective values found (solution quality) averaged over runs, the standard deviation of these outcomes (solution stability), and statistical significance via Wilcoxon rank-sum tests. The results showed that SSA obtained superior or very competitive results compared to the other optimizers on the majority of benchmark functions. In particular, SSA excelled on unimodal problems by quickly converging to the optimum (demonstrating strong exploitation and fast convergence rates). On complex multimodal functions, SSA also outperformed most rivals, indicating a robust ability to explore the search space and avoid many local minima. These performance gains were not only in average solution quality but were also consistent (smaller standard

deviations across runs), highlighting the algorithm's reliability. Statistical tests confirmed that SSA's improvements were significant in most cases (with p–values ¡ 0.05). The paper also applied SSA (and a proposed multi-objective extension, MSSA) to challenging engineering design problems, such as airfoil shape optimization and marine propeller design, to demonstrate its practical effectiveness on real-world problems with unknown and rugged search spaces. SSA successfully found high-quality solutions for these engineering cases, and the multi-objective version (MSSA) was able to approximate Pareto-optimal fronts with good convergence and coverage across objectives.

The strengths of SSA, as identified in the paper, include its balanced exploration-exploitation strategy and swarm dynamics that help prevent premature convergence. The adaptive coefficient c_1 gradually shifts the search from exploration in early iterations to intensification in later stages, enabling SSA to first broadly sample the search space and then hone in on optima. The chain-like follower update mechanism means individuals share information and collectively move toward the best region found, which helps the swarm escape local optima traps more effectively than many evolutionary algorithms. Accordingly, SSA showed both faster convergence and more robust avoidance of local minima compared to classical algorithms like PSO and GA on the test problems. Another advantage is that SSA is conceptually simple and requires only a few control parameters that were fixed in the paper (population size and iterations), yet it generalizes well across different problems. The authors also note that SSA's performance does not significantly degrade when scaling up the problem dimensionality, at least for moderate dimensions; in experiments from 50 up to 200 decision variables, SSA maintained a high solution quality with only a mild decrease as dimensionality grew, indicating good scalability given a sufficient population

size. Despite these merits, the paper acknowledges some limitations. Like any stochastic optimizer, SSA cannot guarantee superior performance on every possible problem (as per the No Free Lunch theorem), and there may be certain landscapes or higher-dimensional scenarios where tuning or hybridization is needed to maintain efficiency. Also, the original SSA is designed for single-objective optimization – it retains only one best-so-far solution – which means it cannot directly produce a diverse set of trade-off solutions for multi-objective tasks. This limitation was addressed by the authors through the Multi-objective SSA (MSSA) that incorporates mechanisms (inspired by Pareto dominance and archiving, analogous to NSGA-II) to handle multiple objectives. In summary, the Salp Swarm Algorithm emerges from this study as a powerful bio-inspired optimizer that leverages a unique inspiration (salp swarming chains) and a mathematically elegant update scheme to achieve competitive results. It offers fast convergence, stable outcomes, and effective global search behavior on a wide range of benchmark functions and engineering problems. The original authors report that SSA often outperforms or at least matches the best of contemporary algorithms (such as PSO, GSA, BA, FA, etc.) on tested problems, while being statistically reliable, thus making it a promising addition to the family of nature-inspired metaheuristics for complex optimization tasks. Future research suggested in the paper includes further investigating SSA's parameters, hybridizing it with other techniques, and exploring its application to more real-world and high-dimensional problems, to fully exploit its exploration–exploitation balance in new domains.

Each of the above algorithms has been applied to optimization problems and some specifically to feature selection, often showing competitive performance in benchmark studies. In this work, all 13 algorithms above are implemented in binary form (using common transfer functions) to select

features for credit scoring models, and their performance is systematically compared.

2.5 Related Work

Several studies have investigated the application of metaheuristic algorithms for feature selection using wrapper-based approaches.

In Barrera-García et al. [1] conducted a systematic literature review of metaheuristic feature selection studies published between 2019 and 2023, with an emphasis on problem formulation and evaluation methodologies. Analyzing 161 recent papers, the authors investigate how feature selection objectives are defined and measured, and they propose a new classification scheme for common objective functions and performance metrics used in this field. They provide an in-depth overview of the metaheuristic techniques most frequently employed for feature subset optimization, along with the benchmark datasets and real-world application areas (such as bioinformatics and text classification) where these techniques have been applied. This review synthesizes emerging trends in feature selection research and offers insights into future opportunities, calling for improved evaluation standards and exploration of under-researched metaheuristic strategies.

In Alyasiri et al. [2] performed a systematic review of wrapper and hybrid feature selection methods using metaheuristic algorithms for text classification in English. The authors surveyed 37 studies from 2016–2021, classifying them into pure wrapper approaches and hybrid approaches that combine filters with metaheuristic wrappers. They further sub-categorize the wrapper-

based methods as adapted, modified, or hybridized versions of standard metaheuristics, and discuss how these techniques have been tailored to improve text classification accuracy while mitigating computational cost. The review finds that purely wrapper-based feature selection can be computationally intensive due to the large search space, whereas hybrid approaches (which often start with a filtering phase) help reduce dimensionality and search time. The authors report that hybrid metaheuristic feature selectors generally achieve better trade-offs between accuracy and efficiency in text classification, and they note that research on such hybrid methods for languages other than English is still limited. Key findings include summaries of how each metaheuristic (e.g., genetic algorithms, particle swarm optimization) was applied to text corpora and the resulting improvements, and the paper concludes with identified gaps and recommendations for future research in text feature selection.

In Kaur et al. [3] presented a systematic review of metaheuristic optimization techniques applied to feature selection in medical disease diagnosis. Covering 173 papers across multiple scholarly databases, the authors examine ten nature-inspired metaheuristic algorithms – including Spider Monkey, Shuffled Frog-Leaping, Cuckoo Search, Ant Lion Optimizer, Lion Algorithm, Moth-Flame Optimizer, Bat Algorithm, Grey Wolf Optimizer, Whale Optimization, and Dragonfly algorithm – and how they have been used to select important clinical features for disease prediction. The review spans a range of diseases (heart disease, Alzheimer's, diabetes, liver disease, COVID-19, etc.), summarizing each study's methodology and the metaheuristic's role in improving diagnostic accuracy. The authors outline a generic framework for applying metaheuristic feature selection in healthcare, detailing the typical phases (preprocessing, feature optimization, and classification) involved in

these studies. They also compare the performance of different approaches using evaluation metrics like accuracy, precision, F1-score, sensitivity, and specificity. The key contribution of this work is identifying open issues and challenges in the field (such as handling class imbalance and selecting algorithm parameters for medical data) and providing guidance for researchers on choosing suitable metaheuristic techniques for future disease prediction problems.

In Agrawal et al. [4] conducted an extensive survey of metaheuristic algorithms for feature selection, reviewing research from 2009–2019 on binary variants of nature-inspired optimizers for classification problems. The authors categorized over a hundred algorithms by their behavioral inspiration (e.g., evolutionary, swarm-based, etc.) and documented the classifiers, benchmark datasets (primarily from the UCI repository), and evaluation metrics used in each study. They identified that metaheuristic-based feature selection gained prominence due to its ability to find compact, informative feature subsets that improve classification accuracy. The survey also highlighted common challenges in obtaining optimal feature subsets and outlined research gaps, concluding with a case study comparing several algorithms on standard datasets to guide future developments in this field.

In Dokeroglu et al. [5] presented a comprehensive survey of recent metaheuristic feature selection algorithms, focusing on prominent methods from the last two decades. This work examines how modern metaheuristics handle the feature selection search space, analyzing their exploration–exploitation balance, selection operators, transfer functions for binary representation, fitness evaluation strategies, and parameter tuning techniques. The authors discuss numerous successful applications of these algorithms in machine learn-

ing tasks with high-dimensional data, where feature selection is crucial for improving classifier performance. They summarize current challenges (such as balancing search intensification and diversification) and suggest future research directions to enhance the effectiveness of metaheuristic feature selection approaches.

In Nssibi et al. [6] provided a comprehensive review of the feature selection process and its integration with nature-inspired metaheuristic optimization techniques. The authors propose a taxonomic framework that organizes feature selection methods and metaheuristic variants (including binary, continuous, island-based, chaotic, and fractional-chaotic versions) and thoroughly explain different search strategies and evaluation criteria. Covering a broad range of application domains (from text mining to engineering and medical diagnosis), this survey emphasizes the importance of representation schemes and fitness functions in designing effective feature selection algorithms. The review addresses the lack of prior unified analyses by examining the strengths and limitations of various approaches, and it outlines open issues and promising research directions for advancing metaheuristic-based feature selection

In Akinola et al. [7] presented a systematic survey focused on multiclass feature selection problems tackled by metaheuristic algorithms. Noting that most feature selection research addresses binary classification, the authors collected studies from 2000 to 2022 that specifically handle datasets with multiple classes. They categorize metaheuristic techniques into four groups based on their nature (evolutionary-based, swarm-intelligence, physics-based, and human-inspired) and list representative algorithms in each category. The survey also reviews common application domains for multiclass feature se-

lection (including image recognition and medical diagnosis) and highlights the classifiers frequently paired with these methods to evaluate performance. In addition, the authors discuss the unique challenges of multiclass feature selection, such as maintaining class discrimination across many classes, and identify open research gaps and future directions for developing more efficient metaheuristic solutions in this context.

In Arun Kumar et al. [8] offered a concise survey of metaheuristic-based feature selection techniques, with particular context given to healthcare data and recent literature. The paper provides an overview of prominent metaheuristic approaches for feature selection, describing their key mechanisms and how they eliminate redundant or irrelevant features to improve predictive modeling. The authors highlight several research works published in the last three years (approximately 2019–2021), summarizing each study's proposed methodology, its benefits in terms of improved classification or reduction in feature set size, and any noted limitations. Through this review of recent advances — many of which involve hybrid or improved metaheuristic algorithms tested on medical record datasets — the authors observe that despite the introduction of novel techniques, there remain many open challenges in feature selection. They specifically point out that issues like optimizing computational efficiency, handling noisy data, and improving result interpretability are still not fully resolved, suggesting that further enhancements and research are needed in the metaheuristic feature selection domain.

In Wang et al. [9], a novel quantum-inspired grasshopper optimization algorithm for feature selection was proposed to improve classification performance. In this work, the standard Grasshopper Optimization Algorithm (GOA) is enhanced by integrating quantum computing principles, resulting

in a dynamic quantum binary GOA (called DQBGOA_MR) that uses mutual information and rough set theory to evaluate feature subsets. The quantum aspect expands the search space through probabilistic representation of candidate solutions, helping the algorithm avoid premature convergence and maintain a good exploration–exploitation balance. The authors tested DQBGOA_MR on twenty diverse UCI machine learning datasets, comparing its feature selection efficacy against other swarm intelligence algorithms. Experimental results showed that the proposed quantum GOA variant achieved higher classification accuracies with fewer features selected, and produced more stable results across multiple runs. The study's findings demonstrate that introducing quantum mechanisms into metaheuristic search can yield superior optimization of feature subsets for classification tasks.

In Sheth et al. [10] developed a multi-objective feature selection algorithm using an evolutionary technique for improving medical diagnostics. The authors converted the continuous Jaya optimization algorithm into a binary version to simultaneously minimize two objectives: the classification error rate and the number of features selected. Their approach includes mechanisms to avoid premature convergence (stagnation in local optima) and to balance the trade-off between sensitivity and specificity, which is critical in clinical decision support. The proposed binary Jaya-based method was evaluated on five clinical datasets (lung cancer, breast cancer, diabetes, fertility, and immunotherapy) and further validated against several benchmark datasets from the literature. Compared to other feature selection techniques, this algorithm achieved higher classification accuracy and selected more compact feature sets, as evidenced by improvements in average accuracy, sensitivity, specificity, and computation time. By effectively reducing feature dimensionality while maintaining diagnostic performance, this work contributes

a promising optimization approach for feature selection in healthcare data analysis.

These reviews collectively underscore the significance of wrapper-based feature selection and support the development of more robust and generalizable metaheuristic approaches.

Chapter 3

Methodology

This study uses seven publicly available credit scoring datasets that cover diverse settings (countries, data sizes, and feature dimensions). Each is commonly used in financial risk modeling research. The datasets are:

3.1 Dataset Overview

This study utilizes seven publicly available credit scoring datasets that vary in origin, dimensionality, and class distribution. These datasets are frequently used in financial risk modeling research and provide diverse challenges for feature selection and classification. A brief description of each dataset is given below:

Table 3.1: Datasets used for feature selection experiments

Dataset	**Number of Features**	**Number of Instances**	**Number of Classes**
Australian	14	690	2
German	20	1000	2
HMEQ	12	5960	2
Taiwan	23	30000	2
Japan	15	1250	2
CRX	15	690	2
Loan (Corrected)	13	614	2

These datasets (detailed in Table 3.1) are available from standard repositories (e.g. UCI Machine Learning Repository) and have been used in prior credit scoring studies. Each dataset contains a mix of categorical and numerical features; categorical features were one-hot encoded as needed. The class label indicates credit risk (good/bad credit or repayment default). No further imputation was needed since datasets are publicly curated.

3.2 Mirage Search Optimization (MSO) Algorithm

Overview: MSO is a nature-inspired population-based optimizer motivated by optical mirage phenomena. It operates with a population of candidate solutions X_i ($i = 1, 2, \ldots, N$), each a D-dimensional vector. MSO updates each candidate in two stages per iteration: a superior mirage strategy (global ex-

ploration) followed by an inferior mirage strategy (local exploitation). Both strategies generate new candidate positions via mathematical update rules, and each uses a greedy acceptance (selection) – i.e. the candidate's position is replaced only if the new position yields improved fitness. Key notations include: t for the iteration index; $X_i(t) = (x_{i,1}, \ldots, x_{i,D})$ for the ith solution's position at iteration t; $f(X)$ for the objective function value (fitness); $X_{\text{best}}(t)$ for the best solution in the population at iteration t; and $X_{\text{worst}}(t)$ for the worst solution. All $x_{i,d}$ are constrained to the search bounds $[lb_d, ; ub_d]$ for each dimension d. Randomization is introduced by r, r_1, r_2 which are independent uniform random numbers in $[0, 1]$, and by an occasional discrete random factor $\Lambda \in 1, 2$ for simulating mirage effects as described below. Table 3.2 summarizes the key parameters and their meanings for MSO.

Superior mirage strategy (global exploration): In this first update phase, each candidate is attracted toward the best solution X_{best} but with a mirage-inspired twist to boost exploration. In physical mirages, a "superior" mirage appears above the true object due to atmospheric refraction; analogously, MSO simulates a possibility of overshooting beyond the current best position. For each dimension d, the superior-mirage update is modeled as:

$$x_{i,d}^{\text{sup}}(t+1); =; x_{i,d}(t); +; r_1 \big[, X_{\text{best},d}(t); -; \Lambda \cdot x_{i,d}(t), \big] \ , \tag{3.1}$$

where $x_{i,d}^{\text{sup}}(t+1)$ is the dth component of the new position candidate for the ith solution after the superior strategy. In Eq. (3.1), $r_1 \in [0, 1]$ is a uniform random scalar and $\Lambda \in 1, 2$ is a discrete random parameter. Setting $\Lambda = 1$ simulates the "normal" case where the candidate moves toward the best solution, i.e. $x_{i,d}(t) + r_1(X_{\text{best},d} - x_{i,d})$, which brings X_i closer to X_{best}. Setting $\Lambda = 2$ simulates an "above-horizon" mirage effect: the term becomes

Table 3.2: Key notation and parameters in MSO.

Symbol	**Description**
N	Population size (number of candidate solutions)
D	Problem dimension (number of decision variables)
$X_i(t)$	Position vector of ith candidate at iteration t, $X_i(t) = (x_{i,1}, \ldots, x_{i,D})$
$f(X)$	Objective function (fitness) value of solution X (to be minimized, w.l.o.g.)
$X_{\text{best}}(t)$	Best solution found up to iteration t (global best in population)
$X_{\text{worst}}(t)$	Worst solution in the population at iteration t (highest f-value)
$r,\ r_1,\ r_2$	Uniform random numbers in $[0, 1]$ (drawn anew for each use)
Λ	Mirage orientation factor (randomly 1 or 2) controlling direction of update
$lb_d,\ ub_d$	Lower and upper bound of search space in dimension d

$x_{i,d} + r_1(X_{\text{best},d} - 2x_{i,d}) = x_{i,d} - r_1(x_{i,d} - X_{\text{best},d})$. In this case the update can overshoot beyond X_{best}, enhancing exploration of new regions beyond the current best. Each X_i applies a random choice of Λ for each update, so some moves are direct towards the best ($\Lambda = 1$) while others overshoot ($\Lambda = 2$), diversifying the search.

After computing $X_i^{\text{sup}}(t+1)$ via Eq. (3.1) for a given i, MSO evaluates its fitness $F_i^{\text{sup}} = f(X_i^{\text{sup}}(t+1))$. A greedy selection then retains the better solution between the new and old positions

$$X_i(t) := \begin{cases} X_i^{\text{sup}}(t+1), & \text{if } F_i^{\text{sup}} < f(X_i(t)), \\ X_i(t), & \text{otherwise.} \end{cases} \tag{3.2}$$

where $X_i(t)$ on the left-hand side is updated to the new position only if it has better (lower) objective value. This greedy replacement ensures that MSO never degrades a solution in this step (it "retains only optimal candidate solutions" from the trial updates.

Inferior mirage strategy (local exploitation): In the second phase, MSO refines solutions by a more localized search. The term "inferior mirage" in optics refers to an inverted image below the real object (such as the classic road mirage). MSO's inferior strategy mimics this by focusing on each solution's immediate vicinity and "reflecting" certain positional information to fine-tune the search locally. One way to model this is to drive solutions away from the worst conditions (analogous to avoiding the misleading inverted image). For example, the following update rule can be applied:

$$x_{i,d}^{\text{inf}}(t+1); =; x_{i,d}(t); +; r_2 \big[, x_{i,d}(t); -; X_{\text{worst},d}(t), \big] \ , \tag{3.3}$$

where $x_{i,d}^{\text{inf}}(t+1)$ is the dth component of the new position after the inferior strategy, and $r_2 \in [0,1]$ is a uniform random factor. Eq. (3.3) pushes the ith solution away from the worst solution X_{worst} along each dimension. Intuitively, this encourages even the poorer candidates to move in directions that avoid the known worst areas of the search space, thereby exploiting the region between their current position and better areas. The step size is random and relatively small (bounded by the distance to the worst), so this acts as a localized exploitation move. (In practice, alternative formulations

of the inferior mirage strategy exist; for example, one could use a fraction of the distance toward the best or a neighborhood-best solution for exploitation. The key idea is that the inferior strategy performs fine-grained adjustments to candidates, complementing the exploratory superior step.)

As in the first phase, a greedy acceptance criterion is applied: if the new position $X_i^{\text{inf}}(t+1)$ found by Eq. (3.3) yields a better fitness $F_i^{\text{inf}} = f(X_i^{\text{inf}}(t+1))$ than the current $X_i(t)$, the candidate is updated:

$$X_i(t) := \begin{cases} X_i^{\text{inf}}(t+1), & \text{if } F_i^{\text{inf}} < f(X_i(t)), \\ X_i(t), & \text{otherwise.} \end{cases} \tag{3.4}$$

After both the superior and inferior updates (with their respective greedy selections) have been applied to all candidates in iteration t, the population is ready for the next iteration $t+1$. During the updates, any new position that violates a bound is corrected (clamped) to the domain limits: for each $x_{i,d}$ we enforce $x_{i,d} \in [lb_d, , ub_d]$. In practice this can be done with:

$$x_{i,d}(t+1) := \begin{cases} lb_d, & \text{if } x_{i,d}(t+1) < lb_d, \\ ub_d, & \text{if } x_{i,d}(t+1) > ub_d, \\ x_{i,d}(t+1), & \text{otherwise.} \end{cases} \tag{3.5}$$

for each i and d. This ensures all candidates remain in feasible range. If the optimization problem has additional constraints (e.g. equality/inequality constraints), MSO can incorporate them using penalty functions. For example, a common approach is to add a large penalty term to $f(X)$ when constraints are violated. Let $n_q(X)$ be the number of constraint violations by solution X and let P be a large constant penalty factor; one can redefine

the fitness with a penalty as $f^(X) = f(X) + P \cdot n_q(X)$, so that any infeasible solution is effectively worsened in fitness by an amount proportional to violations. MSO then works on $f^(X)$ in its comparisons, thereby discouraging infeasible moves.

Mathematical summary: Eqs.(3.1)–(3.5) constitute the core mathematical model of MSO's update rules. In summary: each candidate X_i generates a "superior mirage" candidate position X_i^{sup} via Eq.(3.1) (moving toward/around the global best); if X_i^{sup} is better, it replaces X_i. Then X_i (updated if applicable) generates an "inferior mirage" candidate X_i^{inf} via Eq. (3.3) (a localized refinement move); if X_i^{inf} is better it replaces X_i. After these two greedy updates, the algorithm proceeds to the next iteration. This two-step update balances exploration and exploitation. The random factors r_1, r_2, and Λ ensure stochasticity in both phases, helping MSO to avoid purely deterministic moves and thus reducing the chance of stagnation in local optima. Over successive iterations, the population tends to converge as inferior (exploitation) steps fine-tune solutions around promising regions identified by the superior (exploration) steps.

Pseudocode for MSO: Algorithm 1 provides a step-by-step pseudocode for the MSO procedure, encapsulating the above-described process.

Algorithm 1: Pseudocode of the Mirage Search Optimization (MSO) algorithm

Input: Population size N, max iterations T, search bounds $[lb, ub]^D$

Output: Best solution X_{best} and its fitness $f(X_{\text{best}})$

```
Initialize a population of N candidate solutions X_1, ..., X_N in [lb, ub]^D;
Evaluate fitness f(X_i) for all i;
Set t = 0, identify initial X_best(0);
while termination condition not met and t < T do
    t ← t + 1;
    // Superior Mirage Strategy (Exploration)
    foreach X_i in population do
        Generate X_i^sup using Eq. (3.1) relative to X_best(t−1);
        Enforce bounds on X_i^sup using Eq. (3.5);
        Compute F_i^sup = f(X_i^sup);
        if F_i^sup < f(X_i) then
            X_i ← X_i^sup;
    Update X_best if improved;
    // Inferior Mirage Strategy (Exploitation)
    foreach X_i in population do
        Generate X_i^inf using Eq. (3.3) relative to X_worst(t−1);
        Enforce bounds on X_i^inf using Eq. (3.5);
        Compute F_i^inf = f(X_i^inf);
        if F_i^inf < f(X_i) then
            X_i ← X_i^inf;
    Update X_best and X_worst for iteration t;
return X_best and f(X_best)
```

Flowchart of MSO: For clarity, Figure 3.1 illustrates the flow of the MSO algorithm in a diagram format. The process begins with initialization of the population and parameters. In each iteration, the loop executes two main phases (superior mirage update followed by inferior mirage update for each candidate), with fitness evaluations and greedy replacements at each step. The best solution is tracked throughout. The loop continues until a stopping criterion is met (such as a maximum number of iterations or a satisfactory fitness threshold), after which MSO terminates and returns the best solution obtained.

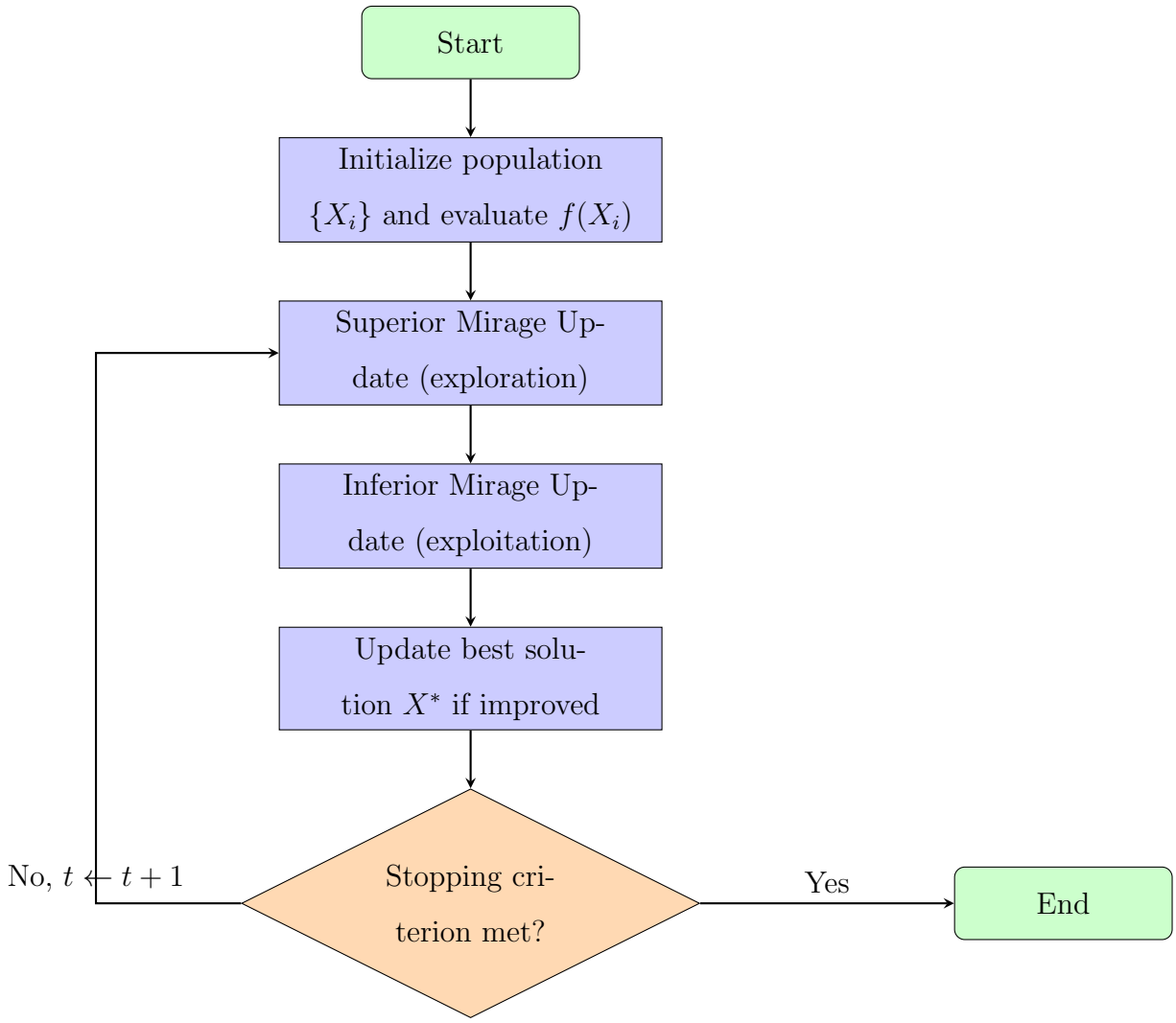

Figure 3.1: Flowchart of the Mirage Search Optimization (MSO) algorithm showing initialization, exploration and exploitation updates, best solution tracking, and termination.

As shown in Figure 3.1, MSO iterates the two-phase update for each solution until termination. The superior mirage strategy provides global search capability by moving solutions toward or beyond the current best, while the inferior mirage strategy provides local search by fine-tuning around the current position (avoiding bad regions). Through these coupled steps, the population converges toward optimal solutions over time.

3.3 Golf Optimization Algorithm (GOA)

Overview: The Golf Optimization Algorithm (GOA) is a game-inspired metaheuristic that mimics the process of playing a game of golf, with a particular focus on the distinct strategies used in different phases of the game. GOA divides the search process into two sequential phases each iteration: a strong swing (exploration phase) analogous to the tee-off drive in golf, and a putting phase (exploitation) analogous to the gentle putt on the green. These phases correspond to global exploration and local exploitation, respectively, ensuring GOA balances searching wide areas of the solution space with fine-tuning around promising regions.

In GOA, we similarly denote $X_i(t)$ as the position of the ith member of the population at iteration t, for $i = 1, \ldots, N$. The best solution found so far is considered the "hole" (the target). GOA uses the following core equations for its two-phase update in iteration t

Phase 1 – Exploration (Strong Swing): Each solution takes a "strong swing" toward the hole (X_{best}). The update is modeled as:

$$x_{i,d}^{(P1)}(t+1); =; x_{i,d}(t); +; r, \left[, B_d(t); -; I \cdot x_{i,d}(t), \right] , \tag{3.6}$$

where $B(t) = X_{\text{best}}(t)$ is the current best ("hole") solution, B_d its dth component, and $r \in [0, 1]$ is a uniform random number. The parameter $I \in 1, 2$ is an integer randomly chosen for each update. If $I = 1$, Eq.(3.6) becomes $x_{i,d} + r(B_d - 1 \cdot x_{i,d}) = x_{i,d} + r(B_d - x_{i,d})$, i.e. a move towards the hole. If $I = 2$, the update becomes $x_{i,d} + r(B_d - 2x_{i,d}) = x_{i,d} - r(x_{i,d} - B_d)$, which can overshoot beyond the hole position. This mechanism is designed

to simulate real golf drives where sometimes the ball lands past the hole. It increases exploration diversity: $I = 2$ swings (chosen stochastically) allow searching beyond the current best, while $I = 1$ swings focus on approaching the best. After computing the new candidate $X_i^{(P1)}(t+1)$ via Eq.(3.6), its fitness $F_i^{(P1)}$ is evaluated. GOA then performs a greedy selection similar to MSO:

$$X_i(t) = \begin{cases} X_i^{(P1)}(t+1), & \text{if } F_i^{(P1)} < f(X_i(t)), \\ X_i(t), & \text{otherwise}, \end{cases} \tag{3.7}$$

meaning the new position is kept if it improves the objective value.

Phase 2 – Exploitation (Putt): After the long drive, GOA performs a "putting" action for fine adjustment. The exploitation update uses a smaller, more precise move designed not to overshoot the hole.

Mathematically, for each dimension d:

$$x_{i,d}^{(P2)}(t+1); =; x_{i,d}(t); +; (1-2r); \frac{; lb_d + r, (ub_d - lb_d);}{t} \ , \tag{3.8}$$

where $r \in [0, 1]$ is a uniform random number (independently chosen from the one in phase 1), and lb_d, ub_d are the lower and upper bounds of the search space in the dth dimension. Equation (3.8) can be understood as follows: $lb_d + r(ub_d - lb_d)$ produces a random value in $[lb_d, ; ub_d]$ (a uniformly sampled point in the domain of the dth variable). This is then divided by the iteration count t, which means as t grows, the magnitude of this term shrinks. Multiplying by $(1 - 2r)$ (a random number in $[-1, 1]$) further produces a signed small perturbation. Thus, $x_{i,d}^{(P2)}(t+1)$ is the current position plus a small random offset that becomes ever smaller as iterations progress

(simulating increasingly delicate putts as the algorithm hones in on the target). Early in the search (small t), the term can still be relatively large, allowing moderate-range exploitation; later, it becomes very small, focusing on fine local tuning. After computing $X_i^{(P2)}(t+1)$ by Eq. (3.8), its fitness $F_i^{(P2)}$ is evaluated, and a greedy acceptance is applied:

$$X_i(t) = \begin{cases} X_i^{(P2)}(t+1), & \text{if } F_i^{(P2)} < f(X_i(t)),\ X_i(t), \\ \text{otherwise}, \end{cases} \tag{3.9}$$

replacing X_i with the putt result if it is better. (Note: In GOA's implementation, phases 1 and 2 are applied sequentially to each X_i within the same iteration, much like in MSO's two strategies, so the $X_i(t)$ in Eq. (3.8) would be the possibly already-updated position after phase 1.)

Boundary handling: GOA ensures that any new $x_{i,d}$ produced by Eq.(3.6) or (3.8) stays within $[lb_d, ub_d]$. If a value falls outside, it is set to the nearest boundary (lower bound or upper bound), analogous to Eq.(3.5) earlier. This prevents invalid solutions. GOA can also incorporate constraint handling via penalty functions similar to MSO. In fact, the GOA formulation in the literature uses a penalty term for solutions violating any equality/inequality constraints, adding a large value (e.g. 10^5 times the number of violated constraints) to their objective to discourage them. We omit the explicit formula here (it is conceptually similar to the penalty approach described for MSO).

Variables and parameters in GOA: The population size N, number of iterations T, and search bounds lb_d, ub_d play the same role as in MSO. Additional parameters specific to GOA include the random binary parameter

I controlling overshoot (unique to GOA's exploration step) and the use of iteration counter t in Eq. (3.8) to decay step size. GOA does not explicitly utilize a "worst" solution in its updates; the global best B (hole) is the primary reference in phase 1, and the search space bounds are used in phase 2 to scale the random move

Pseudocode for GOA: The GOA process can be summarized in pseudocode form as shown in Algorithm 2. It closely resembles the MSO pseudocode structure, with the two phases executed in sequence for each population member in each iteration.

Algorithm 2: Pseudocode of the Golf Optimization Algorithm (GOA)

Input: Population size N, max iterations T, search bounds $[lb, ub]^D$

Output: Best solution X_{best} and its fitness $f(X_{\text{best}})$

```
Initialize population X_1, ..., X_N randomly in [lb, ub]^D;
Evaluate f(X_i) for all i;
Set t = 1, determine initial X_best(0) as the best solution in the
  population;
while t ≤ T and not converged do
    foreach X_i, i = 1...N do
        // Phase 1: Exploration (Strong swing)
        Randomly choose I ∈ {1, 2};
        Generate X_i^(P1) using Eq. (3.6);
        Enforce bounds on X_i^(P1);
        Compute F_i^(P1) = f(X_i^(P1));
        if F_i^(P1) < f(X_i) then
            X_i ← X_i^(P1);                          // Eq. (3.7)
        // Phase 2: Exploitation (Putt)
        Generate X_i^(P2) using Eq. (3.8) (using updated X_i);
        Enforce bounds on X_i^(P2);
        Compute F_i^(P2) = f(X_i^(P2));
        if F_i^(P2) < f(X_i) then
            X_i ← X_i^(P2);                          // Eq. (3.9)
    Update X_best(t) as current best in population;
    t ← t + 1;
return X_best and f(X_best)
```

Flowchart of GOA: The flowchart for GOA (Figure3.2) is analogous to MSO's, with the key difference being the internal update formulas in the two phases. In the exploration phase (Phase 1), each solution's position is adjusted by a random vector in the direction of the global best ("hole"), with a chance to overshoot, as per Eq.(3.6). In the exploitation phase (Phase 2), each solution is nudged by a small random offset (Eq.(3.8)) that diminishes over time. Each phase includes a check to retain the better position. This iterative loop continues until the stopping criterion is reached.

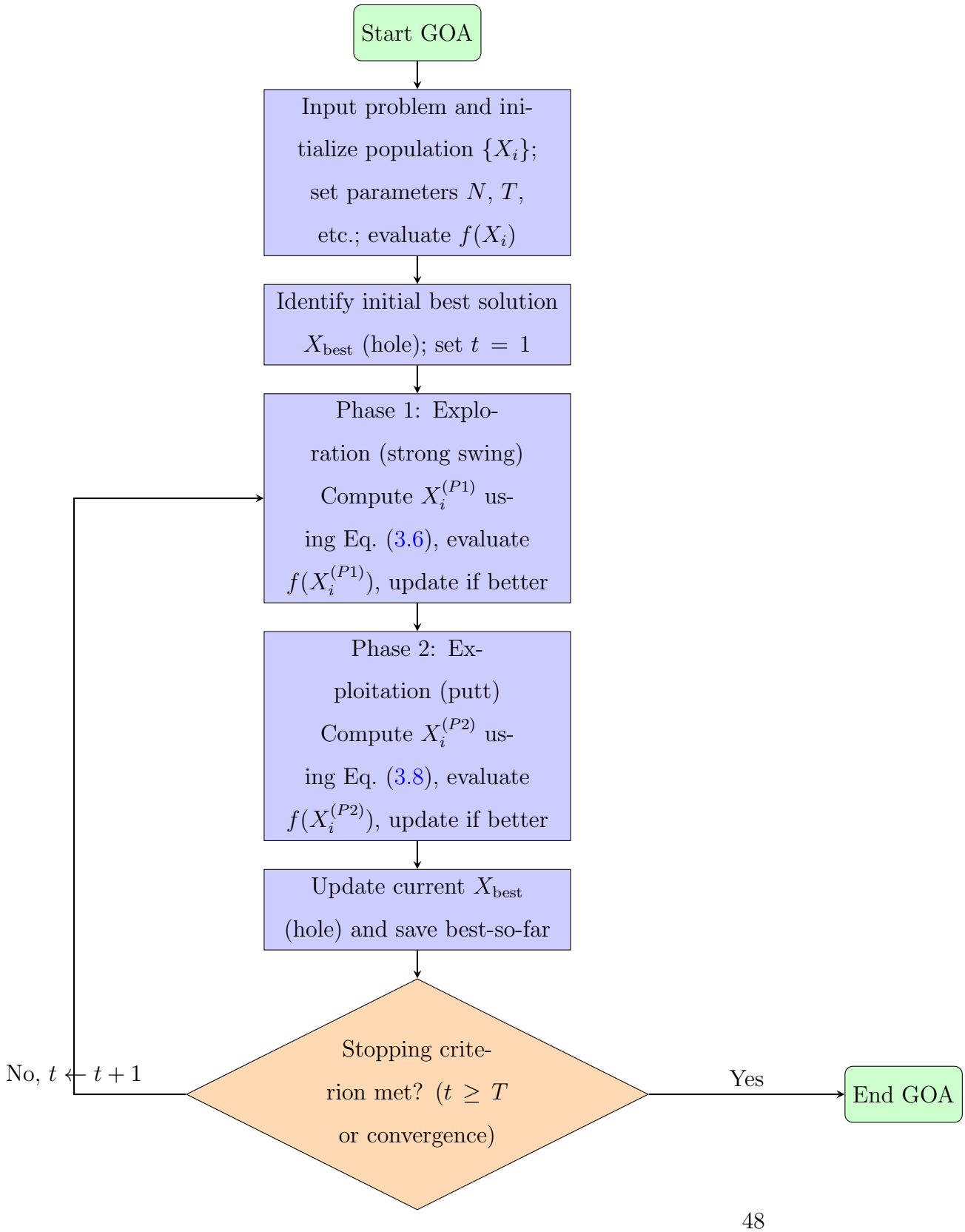

Figure 3.2: Flowchart of the Golf Optimization Algorithm (GOA) showing initialization, exploration (strong swing), exploitation (putt), best solution update, and termination.

In essence, GOA's update rules (Eqs. (3.6)–(3.9)) encapsulate the strategic dichotomy of a golf game: long-range, high-variance moves to cover ground, followed by cautious, precise adjustments to sink the ball. This approach has been shown to provide GOA with a strong balance between exploration and exploitation, as evidenced by its performance on benchmark functions and engineering problems

3.4 Hybrid MSO–GOA Algorithm

Rationale for Hybridization: The hybrid MSO–GOA algorithm is designed to leverage the complementary strengths of MSO and GOA in a single framework. MSO provides a robust two-strategy foundation with proven exploitation capabilities (via its inferior mirage phase and mechanisms like fitness-distance balance, etc.), while GOA contributes an effective exploration operator inspired by the golf swing, which can inject additional diversity into the search. By analogy, here MSO serves as the base algorithm and GOA's exploration phase is incorporated to augment MSO's global search ability. The goal is to attain a superior balance between exploration and exploitation than either algorithm alone, thereby improving convergence to the global optimum.

Hybridization strategy: In the hybrid MSO–GOA, we integrate GOA's Phase 1 update (Eq.(3.6)) into MSO's iteration loop as a replacement or enhancement of MSO's own superior mirage phase. In other words, instead of using MSO's Eq.(3.1) for the exploratory move, we use GOA's Eq.(3.6) (the strong swing toward the current best/hole, with overshoot capability). This injected GOA exploration step is then followed by MSO's inferior mirage ex-

ploitation step (Eq.(3.3)) for local refinement. The combination effectively creates a three-phase update for each candidate in each iteration:

GOA-based exploration: Compute a new candidate position using the GOA exploration formula (strong swing toward the current best solution). This yields $X_i^{\text{GOA}}(t+1) = X_i(t) + r \cdot [X_{\text{best}}(t) - |X_i(t)|]$ (as in Eq. (3.6)). Use greedy selection to keep it if better:

$$X_i'(t+1) = \begin{cases} X_i^{\text{GOA}}(t+1), & \text{if } f\left(X_i^{\text{GOA}}(t+1)\right) < f\left(X_i(t)\right), \\ X_i(t), & \text{otherwise,} \end{cases}$$

where $X_i'(t+1)$ denotes the intermediate updated position after this phase. (Note: $X_i'(t+1)$ will serve as the starting point for the next phase.)

MSO-based exploitation: From $X_i'(t+1)$, perform the MSO inferior mirage update to get a refined candidate: $X_i^{\text{MSO}}(t+1) = X_i'(t+1) + r_2\left[X_i'(t+1) - X_{\text{worst}}(t)\right]$ (as in Eq. (3.3)). Then apply greedy acceptance:

$$X_i^{\text{new}}(t+1) = \begin{cases} X_i^{\text{MSO}}(t+1), & \text{if } f\left(X_i^{\text{MSO}}(t+1)\right) < f\left(X_i'(t+1)\right), \\ X_i'(t+1), & \text{otherwise,} \end{cases}$$

yielding the final updated position $X_i^{\text{new}}(t+1)$ for candidate i at the end of the iteration.

Additional GOA exploitation: In a straightforward implementation, the above two steps are sufficient (the GOA exploitation phase is not explicitly used, since MSO's exploitation takes its role). However, one could optionally also incorporate GOA's fine-grained putt update if desired. For example, after step (2), one might apply Eq. (3.8) to $X_i^{\text{new}}(t+1)$ for an extra local refinement. In our hybrid design, this is typically unnecessary because

MSO's inferior strategy already provides local exploitation. Thus, we generally omit GOA's phase 2 to keep the algorithm efficient, relying on MSO's inferior step for exploitation.

By combining these, each iteration of the hybrid algorithm uses GOA's powerful global exploration move (with potential overshoot via $I = 2$) to broadly explore the search space, then immediately applies MSO's focused exploitation to polish the result. In this algorithm, the base optimizer (MSO) guides the overall search, while the secondary mechanism (GOA's exploration) periodically diversifies the population.

Hybrid pseudocode: The pseudocode in Algorithm 3 summarizes the hybrid MSO–GOA approach. It looks much like MSO's algorithm but with the superior phase replaced by the GOA exploration step:

Algorithm 3: Hybrid MSO–GOA Algorithm: MSO with Injected GOA Exploration

Input: Population size N, maximum iterations T, search bounds $[lb, ub]^D$

Output: Best solution X_{best} and its fitness $f(X_{\text{best}})$

Initialize a population of N candidate solutions $X_1, \ldots, X_N$ in $[lb, ub]^D$;

Evaluate fitness $f(X_i)$ for all i;

Set $t = 0$, and determine initial X_{best} and X_{worst};

while *termination condition not met* ***and*** $t < T$ **do**

  $t \leftarrow t + 1$;

  foreach X_i *in population* **do**

    `// Injected GOA Exploration Phase`

    Randomly select $I \in \{1, 2\}$;

    Compute $X_i^{\text{GOA}} = X_i + r_1 \cdot [X_{\text{best}} - I \cdot X_i]$;

    Enforce bounds on X_i^{GOA};

    Compute $f(X_i^{\text{GOA}})$;

    if $f(X_i^{GOA}) < f(X_i)$ **then**

      $X_i \leftarrow X_i^{\text{GOA}}$ `// Greedy selection`

    `// MSO Inferior Exploitation Phase`

    Compute $X_i^{\text{MSO}} = X_i + r_2 \cdot [X_i - X_{\text{worst}}]$;

    Enforce bounds on X_i^{MSO};

    Compute $f(X_i^{\text{MSO}})$;

    if $f(X_i^{MSO}) < f(X_i)$ **then**

      $X_i \leftarrow X_i^{\text{MSO}}$ `// Greedy selection`

  Update X_{best} and X_{worst} for current population;

  Save current best-so-far solution;

return X_{best} *and* $f(X_{best})$

In Algorithm 3, steps 21–25 implement GOA's strong swing (Eq.(3.6)) as the first part of the loop, and steps 26–29 implement MSO's inferior strategy (Eq.(3.3)) as the second part. By comparing with Algorithm1, one can see that the only major change is replacing the "superior mirage" update with the GOA exploration update. This hybrid loop is depicted in the flowchart of Figure3.3.

Flowchart of Hybrid MSO–GOA: The hybrid algorithm's flowchart (Figure 3.3) is very similar to Figures 3.1 and 3.2, with the exploration step labeled as the GOA-injected operation. Essentially, each iteration does: (a) GOA exploration for each X_i (moving toward current best/hole with random overshoot parameter), then (b) MSO exploitation for each X_i (local mirage refinement away from worst). Greedy selection is applied after each sub-step, ensuring no deterioration. The best and worst are then updated, and the loop repeats.

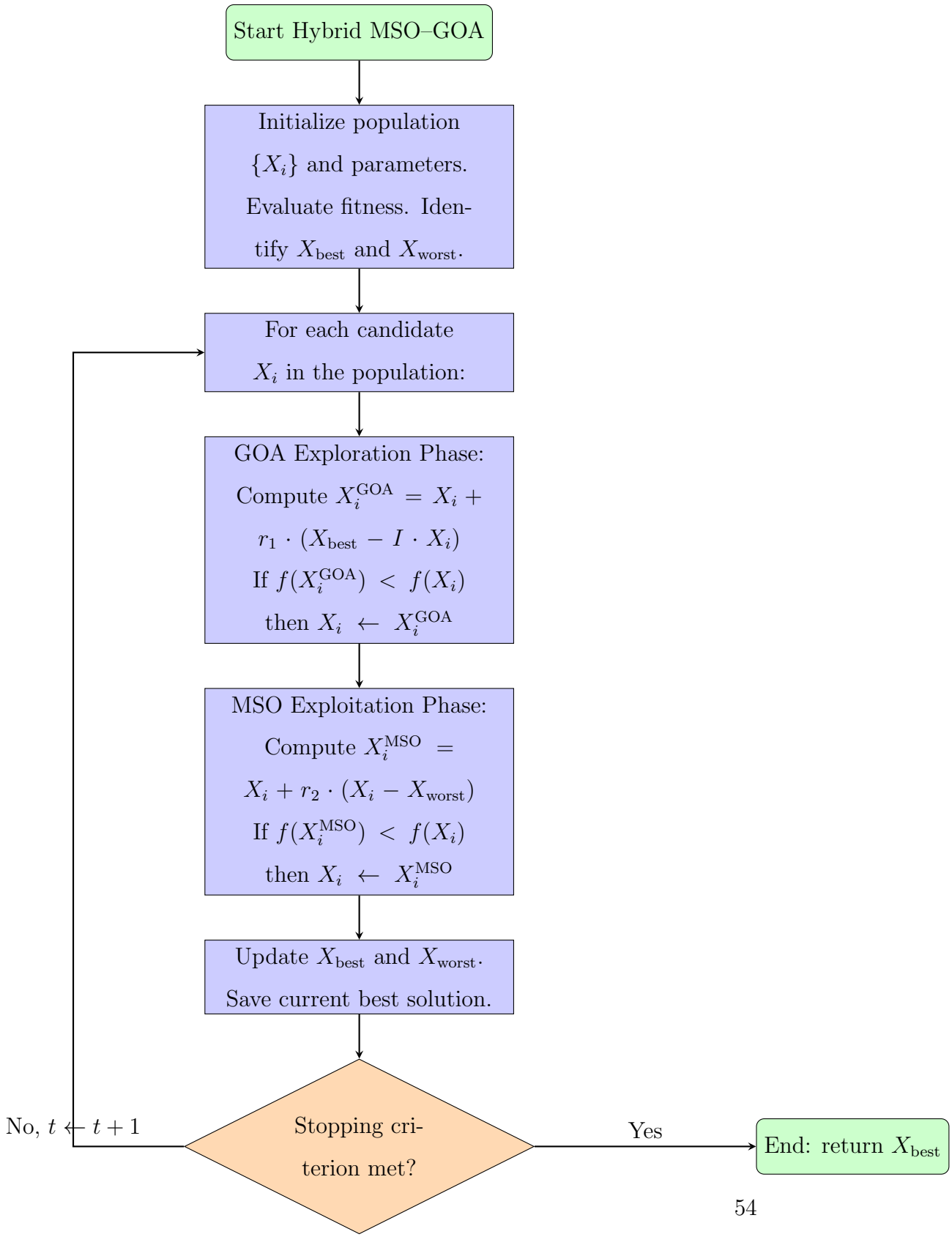

Figure 3.3: Flowchart of the Hybrid MSO–GOA algorithm. Each iteration includes GOA-based exploration followed by MSO-based exploitation, with fitness-based updates and best tracking.

The hybrid MSO–GOA effectively merges the exploration strength of GOA with the exploitation finesse of MSO. GOA's exploration operator introduces additional stochastic diversity – the random overshoot ($I = 2$ cases) can help the population jump out of local optima traps that MSO alone might be caught in. MSO's structured exploitation (especially with mechanisms like fitness-distance constraints if enabled) then takes those diverse candidates and hones them toward feasibility and optimality. In analogy to AGWOCS (where GWO provided structured leadership-based exploitation and CS contributed random exploratory jumps), our hybrid uses MSO as the structured framework and GOA's first phase as the random exploratory injection. This collaboration is expected to yield a more robust search: MSO guides the search with its dual mirage strategies, and whenever intensification needs a boost of diversification, the GOA "strong swing" is there to broaden the search horizon.

Chapter 4

Discussion

4.1 Performance on Standard Benchmark Functions

To rigorously evaluate the effectiveness and convergence ability of the proposed metaheuristic algorithms, a comprehensive benchmark study was performed on a suite of 76 standard optimization functions, widely used in the evolutionary computation literature. These functions span three main categories:

- **Unimodal functions (F1–F7)**: Characterized by a single global optimum, these are primarily used to evaluate the exploitation capability of an algorithm.
- **Multimodal functions (F8–F23)**: Contain multiple local optima, thus assessing the exploration ability and robustness against premature

convergence.

- **Composite functions (F24–F76)**: Represent complex landscapes combining features of both unimodal and multimodal functions, used to simulate real-world challenges and test overall algorithmic performance.

Each of the thirteen evaluated metaheuristic algorithms—including MSO, GOA, and others—was executed independently for 25 runs per function with the same control parameters, population size, and iteration count to ensure fairness. The algorithms were benchmarked based on four core metrics:

Tables 4.1, 4.2, 4.3, and 4.4 provide detailed quantitative results. These highlight that while several algorithms perform well on unimodal problems, their effectiveness deteriorates on multimodal and composite functions, revealing gaps in search space coverage. Notably, MSO and GOA emerged as top performers across a wide range of problems, motivating their selection for hybridization.

Table 4.1: Mean Objective Values of Algorithms (F1–F76)

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F1	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.012994	0.000000	0.000000	0.000000	-0.000000	0.000000	-0.000235
F2	0.000081	0.000000	0.000000	0.000000	0.000000	0.000000	0.000031	1.002052	0.000000	0.000000	-0.000000	0.000000	-0.000046
F3	0.595117	0.000000	0.000000	0.000000	0.000000	0.000000	3602.607334	19.477107	0.000000	23437.727480	0.000000	0.000000	-22.309313
F4	0.084627	0.000000	0.000000	0.000000	0.000000	0.000000	22.803671	6.449546	0.000000	38.997090	-0.000000	0.000000	-16.909405
F5	29.081697	0.000000	0.000000	23.422810	28.719756	25.959411	1396.261812	104.012408	28.116564	27.298487	0.999240	25.285800	-0.438153
F6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.120000	12.880000	0.000000	0.040000	0.004696	0.000000	-0.523298
F7	0.008639	0.000021	0.000011	0.000691	0.001765	0.000040	0.028063	0.085387	0.000065	0.002308	-0.001163	0.000055	-0.196441
F8	-8197.006027	-9372.111630	-3934.658320	-9603.457648	-11973.809890	-7268.844077	-4040.921680	-7613.512185	-3123.776839	-11527.334630	383.858078	-12569.486600	-421.010077
F9	15.904747	0.000000	0.000000	0.000000	0.000000	0.000000	17.745863	43.579152	0.000000	0.000000	-0.000000	0.000000	-2.587031
F10	0.000029	0.000000	0.000000	0.000000	0.000000	0.000000	10.604994	2.046464	0.000000	0.000000	0.000000	0.000000	-0.409580
F11	0.016366	0.000000	0.000000	0.000000	0.000000	0.000000	0.310984	0.009351	0.000000	0.009792	-0.000000	0.000000	-4.912522
F12	0.000000	0.000000	0.015947	0.000000	0.001269	0.004147	1.664970	4.778181	0.137872	0.005516	-0.991861	0.000000	-2.271487
F13	0.000001	0.000000	0.000000	0.000588	0.214190	0.008237	98.991465	0.011654	2.730194	0.245317	0.998804	0.000000	0.919548
F14	0.998004	0.998004	2.001053	0.998004	3.705393	1.157048	1.553806	0.998004	3.242075	1.474509	-31.980148	1.470400	-30.043922
F15	0.001291	0.000976	0.000406	0.000307	0.002339	0.000355	0.000997	0.001490	0.001231	0.000794	0.188817	0.000761	-0.115959
F16	-1.031628	-1.030832	-1.031578	-1.031628	-0.998982	-1.031628	-1.031607	-1.031628	-1.024192	-1.031628	-0.003797	-1.031600	-0.314055
F17	0.397887	0.444155	0.400823	0.397887	0.397887	0.397887	0.399248	0.397887	0.422453	0.397889	4.143535	0.397890	1.255681
F18	3.000000	5.385805	3.049676	3.000000	6.240040	3.000000	3.000020	3.000000	3.698690	3.000006	0.005581	6.240000	-1.000000
F19	-3.862782	-3.647775	-3.860732	-3.862782	-3.862782	-3.862781	-3.854087	-3.862782	-3.698739	-3.859294	0.112118	-3.862800	0.114614
F20	-3.260170	-2.318765	-3.091808	-3.321995	-3.278998	-3.273642	-2.945936	-3.231464	-3.217263	-3.274726	0.209250	-3.293500	0.103464
F21	-6.340174	-8.466804	-6.797651	-10.153200	-9.849978	-10.153197	-1.801892	-9.749005	-4.884865	-8.830855	4.000033	-6.199200	4.759760
F22	-7.643142	-8.416792	-7.515591	-10.402941	-8.151950	-10.402939	-4.012593	-9.673325	-5.455792	-9.448573	4.000589	-5.783800	4.640630
F23	-7.815650	-7.660832	-6.577813	-10.536410	-8.140791	-10.536408	-4.304297	-10.321979	-5.313187	-8.159042	4.000757	-6.630500	3.624780
F24	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000002	0.000000	0.000000	0.000000	0.000000	0.000000	-0.090017

Table 4.1: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F25	0.000863	0.000000	0.000000	0.000000	0.000000	0.000000	0.001140	7.886875	0.000000	0.000000	-0.000000	0.000000	-0.000010
F26	0.076257	0.000000	0.000000	0.000000	0.000000	0.000000	17.206962	7.118702	0.000000	39.089627	-0.000000	0.000000	-16.466945
F27	-151.920000	-154.640000	-111.266667	-155.000000	-155.000000	-132.560000	-102.800000	-134.760000	-118.520000	-155.000000	-5.092991	-155.000000	-5.120000
F28	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	719.784678	0.000000	0.000000	0.000000	0.000000	0.000000	-0.008119
F29	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000014	0.000000	0.000000	0.000000	-0.000000	0.000000	-0.000046
F30	0.666673	0.249446	0.246755	0.666667	0.680608	0.666669	3.174389	0.958193	0.667271	0.666719	0.697813	0.666670	-0.162231
F31	0.000229	0.000000	0.000000	0.000000	0.000000	0.000000	1.620676	0.180168	0.000000	0.000001	-0.000000	0.000000	-2.075760
F32	0.000000	-1.000000	-1.000000	-0.491788	-0.760000	-1.000000	0.000000	0.000000	0.000000	-0.720000	-0.000000	-1.000000	-20.000000
F33	13.258993	15810.437631	483.470642	0.018132	489.896291	15.348557	51.932502	20.497355	730.772258	1371.038881	2.140681	0.774070	1.090658
F34	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.003881	0.003356	0.000000	0.000000	-0.000000	0.000000	-0.000040
F35	0.000000	0.230815	0.019913	0.000000	0.091448	0.000000	0.000148	0.000000	0.005512	0.030483	2.903517	0.000000	0.500000
F36	0.000000	0.084688	0.000009	0.000000	0.000000	0.000000	0.000727	0.000000	0.000417	0.000196	1.011116	0.000000	1.000000
F37	0.000000	0.000000	0.011355	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	-10.000000	0.000000	-10.000000
F38	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
F39	0.292579	0.292580	0.292579	0.292579	0.292861	0.292581	0.292579	0.292579	0.292582	0.292588	-0.247145	0.292580	-1.216196
F40	19.105880	19.485050	19.106249	19.105880	19.106166	19.105880	19.128404	19.105880	19.114721	19.106829	5.133451	19.105900	5.146897
F41	0.000000	0.127878	0.049868	0.000000	0.001091	0.000000	0.000351	0.000000	0.000097	0.000010	0.999894	0.000042	0.988513
F42	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.534065	0.000000	-2.890234
F43	-0.003791	-0.003780	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.029662	-0.003791	-0.029896
F44	137.752669	97.632533	291.068441	93.021594	13.393341	169.439204	290.038401	175.574234	310.459512	93.485924	-384.739596	0.000013	-420.969361
F45	1.008944	0.900000	0.900000	1.002344	0.910738	0.900000	5.158290	1.000000	7.140443	1.063484	-0.000000	0.900000	-9.299120
F46	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
F47	0.477149	2052.876926	1185.474783	7.173242	168.071021	0.153554	402275.034800	0.718777	1362.362318	633.633623	0.073034	0.008716	-5.303421
F48	1.604313	0.000000	0.000000	0.000000	0.000000	0.000000	0.192381	2.566600	0.000000	0.000000	0.000000	0.000000	-4.747174
F49	0.003398	0.000000	0.000000	0.000000	0.000000	0.000000	0.004754	2.110498	0.000000	0.005878	-0.000000	0.000000	-1.537826

Table 4.1: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F50	81.122562	1.000001	1.000001	17.176606	80.156058	28.832738	119.199429	139.211176	94.705098	72.220287	0.730868	1.000000	-3.541378
F51	0.459873	0.000000	0.000000	0.111873	0.000000	0.000000	0.316843	1.007873	0.099873	0.107894	-0.000000	0.000000	-21.426067
F52	-1078.854736	-1117.794918	-849.377854	-1092.992000	-965.231009	-1025.717195	-620.356771	-1005.909811	-614.050429	-1120.279695	-2.899059	-1174.985000	-2.903852
F53	0.000000	-1.000000	-1.000000	0.000000	-1.000000	-1.000000	0.000000	0.000000	0.000000	-0.120000	-0.000000	-1.000000	-3.267277
F54	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.349397	0.000000	-6.266524
F55	-15.674703	-11.855980	-15.302418	-19.831909	-14.945821	-18.614794	-7.361005	-19.414173	-11.200037	-11.645431	1.818931	-27.713600	0.521299
F56	-195.629028	-195.628826	-195.629028	-195.629028	-195.629028	-195.629021	-195.628856	-195.629028	-195.627756	-195.629028	-0.567841	-195.629000	-0.502327
F57	-2.021807	-1.850439	-2.003232	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	2.000000	-2.021800	0.105783
F58	-106.764537	-104.465721	-106.742026	-106.764537	-106.764537	-106.764536	-106.708738	-106.764537	-97.529803	-106.764535	2.179818	-106.764500	-0.740813
F59	-2.062612	-2.062612	-2.062612	-2.062612	-2.062612	-2.062612	-2.062599	-2.062612	-2.061505	-2.062612	0.161929	-2.062600	-0.485786
F60	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000001	1.000000	1.000000	0.000000	1.000000	0.000000
F61	180.327564	5.577404	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	-5.000000	180.327600	-5.000000
F62	-24.156816	-24.155736	-24.153120	-24.156816	-24.156816	-24.156816	-24.113428	-24.156816	-21.420044	-24.156711	4.244314	-24.156800	-6.559394
F63	-42.926498	-42.764692	-42.942950	-42.944387	-42.854944	-42.944384	-42.941005	-42.872833	-42.633924	-42.722620	6.192216	-42.944400	-2.254681
F64	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.053976	0.000048	-0.809644
F65	-0.001549	-0.680242	-0.541405	-0.042205	-0.132510	-0.126994	-0.440114	-0.001025	-0.000204	-0.002333	1.519840	-1.000000	-5.472973
F66	0.126698	0.099770	0.004426	0.036474	0.006549	0.022892	0.204678	0.107085	0.505237	0.188476	4.170656	0.000100	-4.280930
F67	-1.000000	-0.999969	-1.000000	-1.000000	-0.840013	-1.000000	-0.999057	-1.000000	-0.120000	-1.000000	3.141593	-1.000000	3.141593
F68	0.064470	0.064473	0.064471	0.064470	0.064470	0.064470	0.064482	0.064470	0.064471	0.064470	0.467320	0.064470	0.467320
F69	0.000007	0.555453	0.185340	0.000000	1.936631	0.000000	0.002725	0.000000	0.027338	1.490063	0.990770	0.000007	-0.000569
F70	0.000000	0.324957	0.017302	0.000000	0.000000	0.000000	0.008979	0.000000	0.577971	0.000003	-0.453098	0.000000	-0.501532
F71	-19.208503	-19.158950	-19.198847	-19.208503	-19.208503	-19.208502	-19.193209	-19.208503	-17.716536	-19.208503	2.245034	-19.208500	-4.832439
F72	-0.963535	-0.962004	-0.962922	-0.963535	-0.963535	-0.963535	-0.963499	-0.963535	-0.961201	-0.963535	2.700928	-0.963530	-5.016007
F73	-10.872300	-10.872159	-10.872297	-10.872300	-10.824186	-10.872300	-10.872299	-10.870715	-10.842401	-10.866752	-0.063785	-10.872300	-1.569464
F74	-186.730909	-184.669751	-186.706946	-186.730909	-184.204741	-186.730909	-186.319232	-186.730909	-181.305331	-186.730810	0.634091	-186.730900	-3.160833

Table 4.1: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F75	-6.842650	-8.086741	-6.922763	-10.536410	-8.748348	-10.536408	-4.179757	-10.212047	-5.557074	-9.336444	4.000755	-6.000600	3.768876
F76	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

Table 4.2: Best Fit results of 13 Algorithms Across 76 Benchmark Functions

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F1	0	0	0	0.0	0.0	0	0.000042	0.0	0	0.0	-0.0	0	-0.001679
F2	0.000016	0	0	0.0	0.0	0	0.0	0.000748	0.0	0.0	-0.0	0	-0.001096
F3	0.148304	0	0	0.0	0.0	0	229.970085	4.073326	0.0	6231.775924	-0.0	0	-39.558552
F4	0.021102	04.39356274680308e-318		0.0	0.0	0	2.874763	2.648258	0.0	0.003642	-0.0	0	-25.411052
F5	22.999016	0	0	22.594572	28.39529	25.504583	27.890106	25.998608	27.111023	26.545659	0.991026	25.0309	-1.967413
F6	0	0	0	0	0	0	0	5	0	0	-0.330035	0	-1.204626
F7	0.001054	0.000004	0.0	0.000114	0.000016	0.000008	0.004118	0.033101	0.000004	0.000036	-0.015534	0.000001	-0.311706
F8	-9132.413584	-12454.736983	-5234.553834	-10452.75518	-12569.48212	-8629.513834	-4743.790473	-9014.745819	-3828.180942	-12569.20457	-500	-12569.4866	-500
F9	0	0	0	0	0	0	0.000003	20.89413	0	0	-0.0	0	-3.979784
F10	0.000014	0	0.0	0.0	0.0	0.0	0.000238	0.000022	0.0	0.0	-0.0	0.0	-0.94904
F11	0	0	0	0	0	0	0.0	0.0	0	0	-0.0	0	-14.893911
F12	0	0	0.0	0.0	0.000571	0.0	0.434881	1.012427	0.051244	0.000992	-1.011676	0.0	-8.979667
F13	0	0	0.0	0.0	0.026419	0.0	2.262788	0.0	1.664525	0.022153	0.992534	0.0	0.340784
F14	0.998004	0.998004	0.998004	0.998004	0.998004	0.998004	0.998004	0.998004	0.998026	0.998004	-32.008034	0.998	-31.978335
F15	0.000307	0.000329	0.000308	0.000307	0.000349	0.000307	0.000368	0.000309	0.000312	0.000308	0.1757	0.000307	-5
F16	-1.031628	-1.031628	-1.031628	-1.031628	-1.031628	-1.031628	-1.031628	-1.031628	-1.031627	-1.031628	-0.105104	-1.0316	-0.712656
F17	0.397887	0.397889	0.397887	0.397887	0.397887	0.397887	0.397936	0.397887	0.397895	0.397887	3.122639	0.39789	-3.141593
F18	3	3.000663	3	3	3	3.0	3.0	3	3.000396	3	-0.066423	3	-1
F19	-3.862782	-3.855967	-3.862782	-3.862782	-3.862782	-3.862782	-3.854894	-3.862782	-3.855918	-3.862746	0	-3.8628	0.114614
F20	-3.321995	-2.984337	-3.313581	-3.321995	-3.321994	-3.321993	-3.130047	-3.321995	-3.321741	-3.321952	0.178329	-3.322	0.038488
F21	-10.1532	-10.151668	-10.124188	-10.1532	-10.153199	-10.1532	-4.967743	-10.1532	-5.825474	-10.152975	3.999908	-10.1532	1.000132
F22	-10.402941	-10.402085	-10.396439	-10.402941	-10.402941	-10.40294	-9.435534	-10.402941	-10.188419	-10.402934	4.000454	-10.4029	2.004621
F23	-10.53641	-10.483525	-10.534312	-10.53641	-10.53641	-10.53641	-8.302187	-10.53641	-10.028372	-10.536206	4.000642	-10.5364	1.011904
F24	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	-0.0	0	-0.140071

Table 4.2: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F25	0.00031	0	0	0.0	0.0	0	0.000001	0.050914	0.0	0.0	-0.0	0	-0.000169
F26	0.024911	03.71278969339842e-318		0.0	0.0	0	2.480509	1.89908	0.0	0.00002	-0.0	0	-23.47924
F27	-155	-155	-137	-155	-155	-155	-118	-145	-144	-155	-5.12	-155	-5.12
F28	0	0	0	0.0	0.0	0	0.0	0.0	0	0	-0.0	0	-0.016486
F29	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	-0.0	0	-0.000319
F30	0.666667	0.247803	0.215597	0.666667	0.673655	0.666667	0.670858	0.666667	0.666894	0.66667	0.55724	0.66667	-0.497783
F31	0.000025	0	0	0.0	0.0	0	0.000051	0.059602	0.0	0.0	-0.0	0	-4
F32	0	-1	-1	-1	-1	-1	0.0	0.0	0.0	-1	-0.0	-1	-20
F33	0.523263	264.664949	15.045687	0.000581	6.716032	0.20999	7.94671	0.051796	35.479882	13.685645	-0.725491	0.006692	0.409082
F34	0	0	0	0.0	0.0	0	0.000001	0.000004	0	0.0	-0.0	0	-0.000249
F35	0	0.000001	0.0	0	0	0	0.000007	0.0	0.0	0.0	2.651675	0.0	0.5
F36	0	0	0	0	0.0	0	0.00001	0.0	0.000002	0.000007	0.892215	0.0	1
F37	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	-10	0.0	-10
F38	0	0	0	0.0	0.0	0	0.0	0.0	0.0	0	-0.0	0	0
F39	0.292579	0.292579	0.292579	0.292579	0.292579	0.292579	0.292579	0.292579	0.292579	0.292579	-1.253402	0.29258	-3.789205
F40	19.10588	19.106015	19.10588	19.10588	19.10588	19.10588	19.10674	19.10588	19.106015	19.105982	5.002546	19.1059	5.146897
F41	0	0.000007	0.0	0	0.0	0	0.000007	0.0	0.000001	0.0	0.988217	0.0	0.900535
F42	0	0	0	0	0	0	0	0	0	0	-9.642351	0	-9.767023
F43	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.029987	-0.003791	-0.029896
F44	90.889286	0.546053	237.886854	55.477968	0.000341	109.131739	269.824078	127.047265	285.559882	0.04647	-458.701587	0.000013	-420.974072
F45	1.003001	0.9	0.9	1.000053	0.9	0.9	1.926862	1	4.732913	0.9	-0.0	0.9	-9.424829
F46	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	-0.0	0	0
F47	0.014369	1736.855606	475.526748	0.00318	76.337753	0.000154	4642.939367	0.000225	319.856988	135.410547	-2.205242	0.003404	-5.477661
F48	0.000217	0	0	0.0	0.0	0	0.000009	0.392187	0.0	0.0	-0.0	0	-9.324611
F49	0.000001	0	0.0	0.0	0.0	0.0	0.0	0.00035	0.0	0.0	-0.0	0	-4.999049

Table 4.2: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F50	37.179394	1	1	11.621144	59.890669	17.45137	95.073324	83.032682	81.683125	27.841286	-0.256821	1	-10.528003
F51	0.399873	0	0	0.099873	0.0	0	0.099874	0.599873	0.099873	0.0	-0.0	0	-34.000707
F52	-1146.711182	-1166.037783	-1060.732206	-1118.438095	-1067.383287	-1085.466128	-716.651367	-1061.891219	-774.778955	-1174.949489	-3.043331	-1174.985	-2.907588
F53	0	-1	-1	0.0	-1	-1	0.0	0.0	0.0	-1	-0.0	-1	-6.283187
F54	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	-1.251055	0	-6.266616
F55	-19.351694	-13.278542	-18.164597	-23.572137	-18.231733	-23.452126	-8.714437	-24.438058	-12.754641	-13.701956	0	-28.7854	0.000155
F56	-195.629028	-195.629028	-195.629028	-195.629028	-195.629028	-195.629028	-195.629028	-195.629028	-195.629023	-195.629028	-0.692788	-195.629	-0.682577
F57	-2.021807	-2.021114	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	2	-2.0218	0.105783
F58	-106.764537	-106.763183	-106.764537	-106.764537	-106.764537	-106.764537	-106.761326	-106.764537	-106.463657	-106.764537	-1.666374	-106.7645	-3.130247
F59	-2.062612	-2.062612	-2.062612	-2.062612	-2.062612	-2.062612	-2.062611	-2.062612	-2.062603	-2.062612	-1.349407	-2.0626	-1.349407
F60	1	1	1	1	1	1	1	1.0	1	1	-0.0	1	0
F61	180.327564	5.577404	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	-5	180.3276	-5
F62	-24.156816	-24.156816	-24.156816	-24.156816	-24.156816	-24.156816	-24.156034	-24.156816	-24.12947	-24.156816	-9.646168	-24.1568	-9.646168
F63	-42.944387	-42.943432	-42.944387	-42.944387	-42.944387	-42.944387	-42.944347	-42.944387	-42.940233	-42.944381	6.16508	-42.9444	-27.777785
F64	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	-1.349407	0.000048	-1.349407
F65	-0.010059	-1	-1	-1	-0.995035	-1	-1	-0.002922	-0.000255	-0.040832	-9.475017	-1	-9.424778
F66	0.01752	0.0001	0.0001	0.0001	0.001177	0.0001	0.0001	0.037436	0.356602	0.055045	-9.424894	0.0001	-9.424778
F67	-1	-1	-1	-1	-1	-1	-0.999988	-1	-0.999999	-1	3.141593	-1	3.141593
F68	0.06447	0.06447	0.06447	0.06447	0.06447	0.06447	0.06447	0.06447	0.06447	0.06447	0.46732	0.06447	0.46732
F69	0	0.004076	0.0	0.0	0.0	0.0	0.000002	0.0	0.000263	0.000006	0.966172	0.0	-0.179074
F70	0	0.000013	0	0.0	0	0	0.000447	0.0	0.000404	0.0	-3.786007	0.0	-3.77931
F71	-19.208503	-19.208503	-19.208503	-19.208503	-19.208503	-19.208503	-19.208378	-19.208503	-18.947925	-19.208503	-8.092323	-19.2085	-9.66459
F72	-0.963535	-0.963535	-0.963535	-0.963535	-0.963535	-0.963535	-0.963534	-0.963535	-0.963431	-0.963535	-9.646168	-0.96353	-9.646168
F73	-10.8723	-10.8723	-10.8723	-10.8723	-10.8723	-10.8723	-10.8723	-10.8723	-10.872168	-10.8723	-1.575143	-10.8723	-4.707185
F74	-186.730909	-186.730842	-186.730909	-186.730909	-186.730909	-186.730909	-186.719141	-186.730909	-186.710946	-186.730909	-7.083659	-186.7309	-7.708314

Table 4.2: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F75	-10.53641	-10.53388	-10.516813	-10.53641	-10.53641	-10.53641	-8.496713	-10.53641	-10.249797	-10.536126	4.000678	-10.5364	1.000253
F76	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	-0.0	0	0

Table 4.3: Worst Fit results of 13 Algorithms Across 76 Benchmark Functions

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F1	0	0	0	0.0	0.0	0	0.088204	0.0	0	0.0	0.0	0	0.000436
F2	0.000159	0	2.56884197250298e-312	0.0	0.0	0.0	0.000299	4.62667	0.0	0.0	0.0	0	0.000219
F3	1.330594	0	0	0.0	0.0	0	15707.37434	44.939477	0.0	45664.17641	0.0	0	7.728253
F4	0.241473	0	0.0	0.0	0.0	0.0	52.042129	13.439865	0.0	89.878262	0.0	0	4.797982
F5	106.955005	0	0	25.022994	28.855984	26.424432	17588.28345	683.756536	28.834665	28.738318	1.007884	25.517	0.580987
F6	0	0	0	0	0	0	2	21	0	1	0.257249	0	0.197554
F7	0.029049	0.000069	0.000045	0.002338	0.009184	0.000117	0.148067	0.162361	0.000249	0.009569	0.010392	0.000188	0.052161
F8	-6762.699896	-6659.275793	-2851.853566	-8873.351153	-7235.834525	-6034.272698	-3286.192945	-6512.147832	-2658.584518	-7678.948229	423.830065	-12569.4862	98.737403
F9	52.831427	0	0	0	0	0	119.455623	68.652069	0	0	0.0	0	0.642153
F10	0.000052	0	0.0	0.0	0.0	0.0	20.285232	3.681947	0.0	0.0	0.0	0.0	0.469966
F11	0.075423	0	0	0	0	0	0.893913	0.031898	0	0.10058	0.0	0	4.236192
F12	0	0	0.478401	0.0	0.002233	0.103672	10.418206	9.809056	0.297933	0.023866	-0.96308	0.0	2.738827
F13	0.000001	0	0.0	0.013295	1.715144	0.021028	2304.132089	0.054779	3.013719	0.774306	1.006133	0.0	0.171694
F14	0.998004	0.998011	6.903336	0.998004	12.670506	1.992031	2.982105	0.998004	12.670507	2.982105	-31.978073	6.9033	5.303934
F15	0.020363	0.001674	0.001669	0.000307	0.02126	0.001223	0.001514	0.020363	0.015327	0.001593	0.195654	0.001595	1.063038
F16	-1.031628	-1.018296	-1.030783	-1.031628	-0.215464	-1.031628	-1.031482	-1.031628	-0.981832	-1.031628	0.093351	-1.0316	0.305115
F17	0.397887	0.82796	0.443646	0.397887	0.397889	0.397887	0.402868	0.397887	0.622861	0.397911	9.426824	0.39789	2.245756
F18	3	22.088186	4.002294	3	30.000983	3.0	3.000211	3	10.074986	3.00007	0.07405	84	0
F19	-3.862782	-3.160803	-3.818537	-3.862782	-3.862782	-3.862774	-3.851234	-3.862782	-3.343622	-3.84893	0.180397	-3.8628	0
F20	-3.203095	-1.467825	-2.653003	-3.321995	-3.201313	-3.198807	-2.25602	-3.201833	-2.035866	-3.070403	0.399473	-3.2031	0.054147
F21	-2.630472	-5.055196	-5.055196	-10.1532	-2.630472	-10.153189	-0.350654	-5.100772	-2.543749	-2.630416	4.000067	-2.6305	2.086588
F22	-2.765897	-5.087671	-5.087671	-10.402941	-1.837591	-10.402935	-0.907247	-2.751934	-4.302333	-2.765886	4.000734	-2.7519	1.975031
F23	-2.421734	-5.12848	-5.128476	-10.53641	-2.427335	-10.536404	-0.945357	-5.175647	-3.902378	-1.859389	4.000942	-2.4217	1.356205
F24	0	0	0	0.0	0.0	0	0.000022	0.000001	0	0.0	0.0	0	0.033186

Table 4.3: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F25	0.003536	0	3.29770711587178e-315	0.0	0.0	0.0	0.025331	34.261829	0.0	0.0	0.0	0	0.000033
F26	0.193932	0	0.0	0.0	0.0	0.0	34.747191	12.146851	0.0	75.306618	0.0	0	2.983788
F27	-145	-150	-83	-155	-155	-107	-87	-116	-88	-155	-5.010019	-155	0
F28	0	0	0	0.0	0.0	0	14971.82349	0.0	0	0	0.0	0	0.003762
F29	0	0	0	0.0	0.0	0	0.000143	0.0	0	0.0	0.0	0	0.000075
F30	0.666719	0.25	0.25	0.666667	0.690334	0.666677	30.68073	4.500354	0.668257	0.666866	0.870058	0.66667	0.184549
F31	0.000721	0	0	0.0	0.0	0	21.703699	0.465848	0.0	0.00001	0.0	0	1.681599
F32	0	-1	-1	0.0	0.0	-1	0.0	0.0	0.0	0.0	0.0	-1	0
F33	67.007909	323732.243293	3704.471145	0.220818	1738.043719	74.134323	192.624669	96.098309	8030.598283	15478.98725	3.831199	4.715	0.291485
F34	0	0	0	0.0	0.0	0	0.038519	0.018042	0	0.0	0.0	0	0.000053
F35	0	0.887012	0.216553	0	0.76207	0	0.000374	0.0	0.102061	0.76207	3.147131	0.0	0
F36	0	0.529837	0.000254	0	0.000001	0	0.00193	0.0	0.002462	0.000826	1.111293	0.0	0
F37	0	0	0.34064	0.0	0.0	0.0	0.0	0.0	0.0	0.0	-10	0.0	0
F38	0	0	0	0.0	0.0	0	0.0	0.0	0.0	0	0.0	0	0
F39	0.292579	0.292601	0.292579	0.292579	0.294708	0.292586	0.292581	0.292579	0.292599	0.292633	1.253181	0.29258	0.970676
F40	19.10588	23.49683	19.111748	19.10588	19.108843	19.10588	19.186781	19.10588	19.149523	19.110571	5.289604	19.1059	0
F41	0	0.644122	0.17888	0	0.026527	0	0.002264	0.0	0.000303	0.000099	1.017165	0.00056	0.028679
F42	0	0	0	0	0	0	0	0	0	0	9.829642	0	5.018188
F43	-0.003791	-0.003688	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.003791	-0.029054	-0.003791	0
F44	201.643066	194.699964	323.25456	114.869364	76.99275	209.842966	302.262155	207.952908	332.7886	156.075539	317.059041	0.000013	0.001054
F45	1.021477	0.9	0.9	1.022323	1.168447	0.9	7.517096	1	9.03299	1.577984	0.0	0.9	0.628316
F46	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	0.0	0	0
F47	2.40209	2202.133692	3385.208	49.026293	307.867022	2.104426	8017882.034	7.076979	3239.037167	1263.799269	1.871194	0.019065	0.18551
F48	4.88676	0	1.00679067890911e-313	0.0	0.0	0.0	4.001988	7.665255	0.0	0.0	0.0	0	2.108847
F49	0.051322	0	0.0	0.0	0.000001	0.0	0.077972	20.021724	0.0	0.114657	0.0	0	1.329844

Table 4.3: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F50	140.496849	1.000012	1.000005	23.423376	101.006583	49.99404	169.856646	208.110335	108.080237	121.463749	0.938787	1	2.365035
F51	0.599873	0	0	0.199873	0.0	0	0.717246	1.399873	0.099874	0.199873	0.0	0	6.40479
F52	-1019.480744	-1048.29781	-681.036566	-1047.754499	-889.181655	-974.689132	-541.399743	-934.660747	-486.358916	-817.502028	-2.728825	-1174.985	0.000836
F53	0	-1	-1	0.0	-1.0	-1	0.0	0.0	0.0	0.0	0.0	-1	0.628314
F54	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.260428	0	0.000021
F55	-11.132114	-10.85671	-12.207805	-15.27132	-12.068873	-14.350897	-5.854672	-16.640275	-9.747062	-9.843929	2.342048	-26.7383	0.179727
F56	-195.629028	-195.626521	-195.629028	-195.629028	-195.629028	-195.628984	-195.628562	-195.629028	-195.621914	-195.629027	0.694135	-195.629	0.163069
F57	-2.021807	-1.462016	-1.892276	-2.021807	-2.021807	-2.021807	-2.021807	-2.021807	-2.021804	-2.021807	2	-2.0218	0
F58	-106.764537	-95.913612	-106.517998	-106.764537	-106.764537	-106.764531	-106.528808	-106.764537	-81.718302	-106.764518	4.735924	-106.7645	3.011436
F59	-2.062612	-2.062611	-2.062611	-2.062612	-2.062612	-2.062612	-2.062566	-2.062612	-2.05255	-2.062612	1.349407	-2.0626	1.284892
F60	1	1	1	1	1	1	1	1.000013	1	1	0.0	1	0
F61	180.327564	5.577404	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	180.327564	-5	180.3276	0
F62	-24.156816	-24.150129	-24.117391	-24.156816	-24.156816	-24.156815	-24.037513	-24.156816	-20.346334	-24.156111	9.646168	-24.1568	7.218531
F63	-42.497173	-42.479607	-42.924964	-42.944387	-42.497173	-42.944365	-42.891183	-42.497173	-42.237094	-42.497173	6.221816	-42.9444	8.452144
F64	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000048	0.000049	0.000048	1.349407	0.000048	1.101786
F65	-0.000363	-0.000286	-0.00081	-0.000812	-0.000395	-0.002243	-0.000163	-0.000479	-0.00016	-0.000322	9.441426	-1	6.270905
F66	0.253129	0.29948	0.023926	0.09685	0.029082	0.047506	0.56184	0.242639	0.619055	0.314901	9.999951	0.0001	7.132763
F67	-1	-0.999848	-1.0	-1	-0.000081	-1.0	-0.996897	-1	0	-0.999999	3.141593	-1	0
F68	0.06447	0.064483	0.064479	0.06447	0.06447	0.06447	0.06453	0.06447	0.064471	0.06447	0.46732	0.06447	0
F69	0.000026	3.68989	1.458106	0.0	12.21949	0.0	0.029607	0.0	0.385427	9.415596	1.00159	0.000061	0.07827
F70	0	3.198677	0.306361	0.0	0.0	0.0	0.045064	0.0	3.208046	0.000022	3.590929	0.0	2.340689
F71	-19.208503	-18.716446	-18.925367	-19.208503	-19.208503	-19.208502	-19.154184	-19.208503	-16.011421	-19.208503	8.1095	-19.2085	7.425975
F72	-0.963535	-0.950259	-0.957407	-0.963535	-0.963535	-0.963535	-0.963413	-0.963535	-0.954461	-0.963535	9.646197	-0.96353	8.409334
F73	-10.8723	-10.870599	-10.872271	-10.8723	-10.374894	-10.872299	-10.872294	-10.852493	-10.729332	-10.852492	1.577416	-10.8723	2.025752
F74	-186.730909	-157.640251	-186.49613	-186.730909	-123.576771	-186.730909	-183.828807	-186.730909	-141.338308	-186.729616	5.482902	-186.7309	4.293491

Table 4.3: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F75	-1.676553	-5.128478	-5.128473	-10.53641	-2.427335	-10.536406	-0.943779	-2.427335	-4.238544	-2.806627	4.00086	-1.6766	1.114955
F76	0	0	0	0.0	0.0	0	0.0	0.0	0	0.0	0.0	0	0

Table 4.4: Standard Deviation of 13 Algorithms Across 76 Benchmark Functions

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F1	0.000000	0.000000	0.000000e+00	8.760000e-50	1.550000e-25	0.000000e+00	2.518617e-02	2.450000e-09	0.000000e+00	2.480000e-150	0.000000e+00	0.000000e+00	0.000436
F2	0.000034	0.000000	0.000000e+00	1.730000e-27	1.170000e-14	0.000000e+00	6.880000e-05	1.175926e+00	0.000000e+00	2.730000e-102	0.000000e+00	0.000000e+00	0.000219
F3	0.360931	0.000000	0.000000e+00	3.200000e-11	9.120000e-22	0.000000e+00	3.221155e+03	9.933474e+00	6.970000e-80	9.949812e+03	0.000000e+00	0.000000e+00	7.728253
F4	0.054898	0.000000	0.000000e+00	9.930000e-20	8.210000e-14	0.000000e+00	1.295854e+01	3.096705e+00	4.090000e-155	2.953198e+01	0.000000e+00	0.000000e+00	4.797982
F5	16.264552	0.000000	0.000000e+00	5.831664e-01	1.252139e-01	2.466096e-01	3.950977e+03	1.468091e+02	5.983137e-01	6.342958e-01	4.223199e-03	1.272700e-01	0.580987
F6	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	4.396969e-01	4.893193e+00	0.000000e+00	2.000000e-01	1.271826e-01	0.000000e+00	0.197554
F7	0.006002	0.000019	1.190000e-05	4.708830e-04	2.422360e-03	2.875923e-05	3.064810e-02	3.897251e-02	6.160000e-05	2.366533e-03	5.994869e-03	6.221700e-05	0.052161
F8	709.415123	1291.360279	7.167740e+02	3.693387e+02	1.170603e+03	6.361706e+02	3.489686e+02	6.060054e+02	2.850059e+02	1.605096e+03	1.841548e+02	7.949200e-05	98.737403
F9	14.488009	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	3.036250e+01	1.081944e+01	0.000000e+00	0.000000e+00	2.139269e-09	0.000000e+00	0.642153
F10	0.000010	0.000000	0.000000e+00	1.550000e-15	1.380000e-14	0.000000e+00	9.987261e+00	8.217332e-01	0.000000e+00	2.410000e-15	1.209972e-16	0.000000e+00	0.469966
F11	0.020359	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	2.667748e-01	1.095070e-02	0.000000e+00	2.766133e-02	3.084048e-09	0.000000e+00	4.236192
F12	0.000000	0.000000	8.734374e-02	9.330000e-11	4.289730e-04	2.073432e-02	2.116430e+00	2.477027e+00	6.120782e-02	5.636476e-03	1.329489e-02	1.004900e-16	2.738827
F13	0.000000	0.000000	5.570000e-48	2.660751e-03	3.385780e-01	7.070305e-03	4.594912e+02	1.560983e-02	4.219323e-01	1.972551e-01	3.228683e-03	1.598700e-15	0.171694
F14	0.000000	0.000002	1.595368e+00	0.000000e+00	3.685653e+00	3.719309e-01	9.090681e-01	2.230000e-16	2.833160e+00	7.643777e-01	6.136399e-03	1.635100e+00	5.303934
F15	0.003996	0.000484	2.450980e-04	3.270000e-19	5.585917e-03	1.846928e-04	3.858720e-04	3.941834e-03	2.961502e-03	4.250280e-04	5.131395e-03	3.209800e-04	1.063038
F16	0.000000	0.002723	1.889800e-04	6.410000e-16	1.632329e-01	4.490402e-09	2.980000e-05	8.690000e-15	1.325438e-02	2.200000e-10	9.119537e-02	5.067500e-16	0.305115
F17	0.000000	0.095064	1.045526e-02	0.000000e+00	2.730000e-07	0.000000e+00	1.443601e-03	2.170000e-14	4.990335e-02	4.580000e-06	2.349138e+00	0.000000e+00	2.245756
F18	0.000000	4.055082	1.874524e-01	1.110000e-15	8.954997e+00	1.645288e-10	4.680000e-05	9.150000e-14	1.677627e+00	1.500000e-05	4.041848e-02	1.620000e+01	0.000000
F19	0.000000	0.172782	8.057131e-03	2.270000e-15	3.640000e-08	1.988237e-06	9.148260e-04	3.470000e-14	1.462430e-01	3.710655e-03	4.557253e-02	1.631700e-15	0.000000
F20	0.060624	0.390504	1.827611e-01	6.540000e-16	5.840232e-02	6.043491e-02	2.266193e-01	5.192385e-02	2.578580e-01	8.583401e-02	4.127379e-02	5.182400e-02	0.054147
F21	3.317592	1.826313	2.113327e+00	4.130000e-15	1.504107e+00	2.714502e-06	1.616273e+00	1.398954e+00	6.678213e-01	2.439490e+00	3.132275e-05	2.634700e+00	2.086588
F22	3.264040	2.002565	2.306149e+00	3.520000e-15	3.370471e+00	1.231381e-06	2.129491e+00	2.054445e+00	1.451150e+00	2.239214e+00	6.450427e-05	3.055200e+00	1.975031
F23	3.489708	2.027527	2.223266e+00	2.350000e-15	3.582395e+00	1.604309e-06	1.999402e+00	1.072153e+00	1.198403e+00	3.330414e+00	6.625669e-05	3.117800e+00	1.356205
F24	0.000000	0.000000	0.000000e+00	2.570000e-115	3.810000e-52	0.000000e+00	5.280000e-06	1.980000e-07	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.033186

Table 4.4: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F25	0.000645	0.000000	0.000000e+00	5.770000e-27	5.070000e-14	0.000000e+00	5.044512e-03	9.896203e+00	0.000000e+00	1.040000e-101	0.000000e+00	0.000000e+00	0.000033
F26	0.038164	0.000000	0.000000e+00	3.450000e-19	3.160000e-13	0.000000e+00	9.380193e+00	2.526325e+00	2.790000e-155	2.154754e+01	0.000000e+00	0.000000e+00	2.983788
F27	3.134751	1.150362	1.428270e+01	0.000000e+00	0.000000e+00	1.212807e+01	6.843488e+00	6.875561e+00	1.460913e+01	0.000000e+00	3.941303e-02	0.000000e+00	0.000000
F28	0.000000	0.000000	0.000000e+00	0.000000e+00	5.640000e-133	0.000000e+00	3.000192e+03	1.400000e-49	0.000000e+00	0.000000e+00	1.331716e-33	0.000000e+00	0.003762
F29	0.000000	0.000000	0.000000e+00	8.010000e-52	2.440000e-30	0.000000e+00	3.660000e-05	6.880000e-12	0.000000e+00	1.250000e-155	0.000000e+00	0.000000e+00	0.000075
F30	0.000011	0.000410	7.256666e-03	2.950000e-09	4.948021e-03	1.851281e-06	6.757592e+00	7.753208e-01	2.993940e-04	4.770000e-05	7.312628e-02	3.206400e-13	0.184549
F31	0.000180	0.000000	0.000000e+00	5.020000e-17	2.960000e-26	0.000000e+00	5.363428e+00	9.861001e-02	1.570000e-26	3.070000e-06	0.000000e+00	0.000000e+00	1.681599
F32	0.000000	0.000000	0.000000e+00	4.843829e-01	4.358899e-01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	4.582576e-01	5.643835e-10	0.000000e+00	0.000000
F33	16.381941	64245.730132	8.032547e+02	4.307357e-02	4.541391e+02	2.025896e+01	4.406269e+01	2.394107e+01	1.577994e+03	3.123696e+03	1.252907e+00	1.047200e+00	0.291485
F34	0.000000	0.000000	0.000000e+00	6.630000e-51	4.630000e-28	0.000000e+00	8.543546e-03	5.118198e-03	0.000000e+00	3.140000e-147	0.000000e+00	0.000000e+00	0.000053
F35	0.000000	0.263662	4.307735e-02	0.000000e+00	2.527499e-01	0.000000e+00	1.048450e-04	3.490000e-15	2.091175e-02	1.524139e-01	1.254094e-01	1.016400e-19	0.000000
F36	0.000000	0.155029	4.640000e-05	0.000000e+00	3.370000e-07	0.000000e+00	6.444010e-04	1.170000e-14	6.022860e-04	2.130630e-04	5.692446e-02	4.282000e-19	0.000000
F37	0.000000	0.000000	6.219207e-02	4.560000e-103	4.560000e-103	4.556862e-103	4.560000e-103	4.560000e-103	4.560000e-103	4.560000e-103	0.000000e+00	4.556900e-103	0.000000
F38	0.000000	0.000000	0.000000e+00	3.290000e-98	7.890000e-53	0.000000e+00	9.310000e-108	3.540000e-16	0.000000e+00	0.000000e+00	3.849931e-162	0.000000e+00	0.000000
F39	0.000000	0.000004	9.220000e-17	1.340000e-16	6.603220e-04	2.131549e-06	4.900000e-07	3.740000e-14	4.370000e-06	1.380000e-05	6.280966e-01	1.537000e-16	0.970676
F40	0.000000	0.968233	1.163006e-03	6.880000e-15	6.113210e-04	1.708365e-09	2.237584e-02	1.700000e-10	1.071218e-02	1.256273e-03	7.508265e-02	6.445700e-15	0.000000
F41	0.000000	0.168042	5.099288e-02	0.000000e+00	5.300312e-03	0.000000e+00	5.043210e-04	1.530000e-15	8.230000e-05	2.160000e-05	6.150563e-03	1.275400e-04	0.028679
F42	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	6.259834e+00	0.000000e+00	5.018188
F43	0.000000	0.000026	7.060000e-14	1.330000e-18	3.530000e-18	1.267481e-18	3.230000e-10	1.740000e-15	6.740000e-09	2.990000e-09	2.352612e-04	1.091400e-18	0.000000
F44	26.271645	57.256132	2.083562e+01	1.356649e+01	1.820872e+01	2.445299e+01	9.133599e+00	1.975668e+01	1.100477e+01	4.809736e+01	1.523636e+02	2.892200e-12	0.001054
F45	0.004936	0.000000	4.520000e-16	4.630710e-03	5.368938e-02	3.399350e-16	1.592225e+00	2.040000e-11	1.520118e+00	2.163296e-01	8.203862e-10	3.399300e-16	0.628316
F46	0.000000	0.000000	0.000000e+00	1.450000e-138	1.300000e-56	0.000000e+00	2.300000e-136	4.570000e-14	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000
F47	0.567611	128.110242	7.306340e+02	1.352764e+01	6.784256e+01	4.513508e-01	1.603039e+06	1.695670e+00	8.066805e+02	3.030886e+02	1.042568e+00	3.973300e-03	0.185510
F48	1.708803	0.000000	0.000000e+00	2.330000e-28	5.600000e-16	0.000000e+00	7.985464e-01	1.621475e+00	0.000000e+00	1.040000e-101	0.000000e+00	0.000000e+00	2.108847
F49	0.011126	0.000000	1.690000e-38	2.170000e-23	2.240000e-07	7.929175e-114	1.589705e-02	4.029451e+00	9.760000e-09	2.353157e-02	2.962156e-43	0.000000e+00	1.329844

Table 4.4: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F50	29.119106	0.000002	1.300000e-06	3.676147e+00	1.216229e+01	7.106902e+00	2.268685e+01	2.935115e+01	6.700648e+00	2.429753e+01	3.468847e-01	3.372700e-15	2.365035
F51	0.070711	0.000000	0.000000e+00	3.316625e-02	1.070000e-11	0.000000e+00	1.467306e-01	1.800926e-01	5.420000e-08	6.399565e-02	0.000000e+00	0.000000e+00	6.404790
F52	34.867427	31.032849	1.097296e+02	2.080869e+01	3.750336e+01	2.869540e+01	4.013999e+01	3.474298e+01	7.591056e+01	9.013447e+01	7.184507e-02	7.145100e-13	0.000836
F53	0.000000	0.000000	0.000000e+00	1.190000e-20	4.050000e-09	0.000000e+00	1.780000e-10	2.090000e-23	5.220000e-10	3.316625e-01	2.123981e-18	0.000000e+00	0.628314
F54	0.000000	0.000000	9.020000e-12	2.260000e-12	3.790000e-12	4.291004e-11	4.570000e-10	1.380000e-11	3.920000e-13	1.410000e-12	1.222656e+00	0.000000e+00	0.000021
F55	2.049547	0.729199	1.381954e+00	1.725093e+00	1.420900e+00	2.392085e+00	7.266279e-01	2.055639e+00	7.196679e-01	1.078859e+00	7.587080e-01	5.403000e-01	0.179727
F56	0.000000	0.000549	7.880000e-08	5.800000e-14	7.110000e-14	9.985305e-06	1.400830e-04	4.710000e-13	1.993491e-03	2.860000e-07	3.779518e-01	6.486300e-14	0.163069
F57	0.000000	0.161827	3.047578e-02	1.360000e-15	1.390000e-09	8.026043e-09	2.440000e-09	9.060000e-16	5.320000e-07	9.720000e-16	0.000000e+00	1.199200e-15	0.000000
F58	0.000000	3.196986	5.627938e-02	1.420000e-14	7.980000e-14	1.276586e-06	6.345730e-02	2.320000e-13	7.701173e+00	4.040000e-06	3.124408e+00	1.640900e-14	3.011436
F59	0.000000	0.000000	2.000000e-07	9.060000e-16	4.050000e-16	9.619544e-09	1.230000e-05	1.620000e-15	2.503045e-03	1.120000e-10	1.367280e+00	9.064900e-16	1.284892
F60	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	2.670000e-06	0.000000e+00	0.000000e+00	7.496007e-17	0.000000e+00	0.000000
F61	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000
F62	0.000000	0.001805	1.058942e-02	1.100000e-14	2.070000e-13	1.167324e-08	3.141886e-02	9.290000e-14	9.700427e-01	1.639680e-04	8.840859e+00	1.173800e-14	7.218531
F63	0.089443	0.167752	3.965380e-03	2.900000e-14	1.825742e-01	5.433333e-06	1.043930e-02	1.673320e-01	1.925615e-01	2.215999e-01	1.266625e-02	2.748100e-14	8.452144
F64	0.000000	0.000000	4.900000e-14	3.390000e-21	4.120000e-17	5.186982e-13	2.710000e-10	2.790000e-20	2.350000e-08	1.680000e-14	1.376130e+00	9.381300e-21	1.101786
F65	0.002202	0.475735	4.989206e-01	1.995471e-01	2.709433e-01	3.293742e-01	5.065193e-01	5.887630e-04	2.310000e-05	8.044457e-03	8.918559e+00	0.000000e+00	6.270905
F66	0.067603	0.095713	7.572546e-03	2.902127e-02	8.709387e-03	1.536346e-02	2.361911e-01	6.334995e-02	6.244357e-02	6.324114e-02	8.011899e+00	1.383200e-20	7.132763
F67	0.000000	0.000047	3.860000e-09	0.000000e+00	3.741354e-01	3.538736e-15	8.529090e-04	1.090000e-12	3.316615e-01	2.730000e-07	3.186363e-09	0.000000e+00	0.000000
F68	0.000000	0.000004	1.670000e-06	3.970000e-17	4.250000e-17	6.400366e-10	1.640000e-05	6.610000e-17	1.180000e-07	1.410000e-09	4.965444e-09	4.249200e-17	0.000000
F69	0.000008	0.786003	3.451173e-01	2.070000e-30	3.671977e+00	1.394646e-76	6.345595e-03	4.210000e-11	7.967098e-02	2.771535e+00	9.089404e-03	1.303100e-05	0.078270
F70	0.000000	0.686639	5.895312e-02	3.450000e-17	2.890000e-09	3.438565e-31	1.107073e-02	6.630000e-14	8.114487e-01	6.210000e-06	3.031714e+00	9.091000e-18	2.340689
F71	0.000000	0.110420	5.165910e-02	6.960000e-15	6.320000e-15	1.311917e-07	1.259597e-02	6.650000e-14	7.750219e-01	1.770000e-09	7.887589e+00	5.617300e-15	7.425975
F72	0.000000	0.003237	1.200490e-03	0.000000e+00	9.550000e-16	1.331296e-08	2.860000e-05	2.060000e-16	2.472344e-03	5.670000e-08	9.451276e+00	0.000000e+00	8.409334
F73	0.000000	0.000386	7.470000e-06	3.630000e-15	1.124806e-01	2.618882e-07	1.230000e-06	5.484460e-03	3.418532e-02	9.075835e-03	1.600002e+00	3.382100e-15	2.025752
F74	0.000000	6.055355	6.365325e-02	2.540000e-12	1.263083e+01	5.937535e-11	5.941696e-01	1.260000e-11	9.607091e+00	2.940120e-04	3.400606e+00	4.302500e-14	4.293491

Table 4.4: continued from previous page

Function	DO	GAO	GOA	MPA	MSA	PEOA	SCA	SSA	SABO	WOA	BWO	BSLO	MSO
F75	3.969652	1.970901	2.119088e+00	2.290000e-15	3.258852e+00	1.090892e-06	2.307968e+00	1.621815e+00	1.479774e+00	2.431042e+00	4.769719e-05	2.785300e+00	1.114955
F76	0.000000	0.000000	0.000000e+00	9.010000e-128	9.260000e-49	0.000000e+00	8.350000e-133	2.510000e-15	0.000000e+00	1.370000e-154	0.000000e+00	0.000000e+00	0.000000

4.2 Friedman Test for Optimization and Feature Selection

To statistically validate the observed performance differences, the non-parametric Friedman test was applied to both the optimization phase (76 functions) and the feature selection phase (7 datasets $\times$ 3 classifiers). The Friedman test ranks algorithms based on performance on each individual problem, then computes average ranks and significance values.

For optimization, Table 4.5 presents the ranking of all thirteen algorithms across all performance metrics: mean, best, worst, and standard deviation. Results indicate statistically significant performance variation among the algorithms (p-value ¡ 0.05), justifying subsequent comparison and hybrid development.

In the feature selection phase, separate Friedman tests were conducted for each classifier—Logistic Regression (Table 4.6), Naïve Bayes (Table 4.7), and Random Forest (Table 4.8)—across same four metrics: Mean Accuracy, Best Accuracy, Worst Accuracy, and Std. The MSO and GOA consistently ranked among the top across all metrics and classifiers, reinforcing their robustness.

Table 4.5: Friedman Rankings based on Optimization Results

BestFit			**Mean Accuracy**			**WorstFit**			**Standard Deviation**		
Rank	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**
1	MSO	5.17	1	PEOA	5.12	1	BSLO	4.00	1	PEOA	4.52
2	PEOA	5.43	2	MSO	5.13	2	PEOA	5.01	2	BSLO	4.52
3	GOA	6.05	3	BSLO	5.17	3	MPA	5.06	3	MPA	4.86
4	BSLO	6.05	4	MPA	5.53	4	DO	5.58	4	DO	5.62
5	BWO	6.13	5	DO	6.28	5	GOA	6.72	5	GAO	6.02
6	DO	6.34	6	GAO	6.50	6	GAO	6.79	6	GOA	6.06
7	MPA	6.37	7	GOA	6.64	7	BWO	7.63	7	MSA	7.53
8	GAO	6.43	8	WOA	7.66	8	SABO	7.64	8	SSA	7.59
9	MSA	7.20	9	MSA	7.74	9	SSA	7.98	9	WOA	7.71
10	WOA	7.39	10	BWO	7.97	10	MSA	8.23	10	SABO	8.36
11	SSA	8.39	11	SSA	8.03	11	WOA	8.31	11	BWO	8.99
12	SABO	9.43	12	SABO	9.09	12	MSO	8.51	12	MSO	9.30
13	SCA	10.62	13	SCA	10.14	13	SCA	9.53	13	SCA	9.93
(Chi-square) 167.63			*(Chi-square) 161.4*			*(Chi-square) 176.81*			*(Chi-square) 218*		

Table 4.6: Friedman Rankings using Logistic Regression

BestFit			**Mean Accuracy**			**WorstFit**			**Standard Deviation**		
Rank	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**
1	BWO	2.19	1	MSA	2.88	1	MPA	3.75	1	MSA	2.75
2	MSO	5.19	2	BWO	3.88	2	SCA	4.62	2	BWO	4.62
3	MSA	5.25	3	MSO	4.44	3	SSA	5.75	3	DOA	5.31
4	SSA	6.38	4	DOA	6.31	4	WOA	6.00	4	MSO	6.00
5	WOA	7.00	5	PEOA	7.19	5	BSLO	6.00	5	PEOA	7.00
6	DOA	7.31	6	GOA	7.25	6	GAO	6.50	6	GOA	7.31
7	BSLO	7.31	7	BSLO	7.62	7	PEOA	6.88	7	BSLO	7.62
8	SCA	7.44	8	WOA	7.69	8	MSO	7.50	8	SSA	7.75
9	MPA	7.62	9	SSA	7.94	9	GOA	7.88	9	GAO	8.25
10	PEOA	8.44	10	MPA	8.31	10	SABO	7.88	10	SABO	8.25
11	GOA	8.81	11	SABO	9.06	11	DOA	8.38	11	SCA	8.31
12	SABO	9.00	12	SCA	9.19	12	BWO	8.50	12	WOA	8.75
13	GAO	9.06	13	GAO	9.25	13	MSA	11.38	13	MPA	9.06
Chi-square: 23.9577			*Chi-square: 27.1939*			*Chi-square: 23.7857*			*Chi-square: 21.9474*		
p-value: 0.020612			*p-value: 0.007245*			*p-value: 0.02175*			*p-value: 0.038113*		

Table 4.7: Friedman Rankings using Naïve Bayes

BestFit			**Mean Accuracy**			**WorstFit**			**Standard Deviation**		
Rank	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**
1	MSA	1.88	1	MSA	1.25	1	SCA	3.12	1	MSA	1.38
2	MSO	3.62	2	GOA	2.62	2	MPA	4.12	2	GOA	2.75
3	GOA	5.31	3	MSO	4.25	3	WOA	5.12	3	BWO	4.88
4	BSLO	5.75	4	BWO	5.12	4	GAO	5.50	4	MSO	5.00
5	BWO	6.56	5	BSLO	5.38	5	BSLO	6.00	5	SABO	6.75
6	PEOA	6.88	6	SSA	7.25	6	DOA	6.50	6	BSLO	6.88
7	SSA	6.88	7	SABO	7.38	7	PEOA	6.75	7	SSA	7.62
8	DOA	7.50	8	DOA	7.94	8	SABO	7.25	8	PEOA	8.19
9	SABO	9.06	9	PEOA	7.94	9	SSA	7.38	9	DOA	8.56
10	GAO	9.12	10	WOA	9.00	10	MSO	7.50	10	WOA	8.56
11	WOA	9.12	11	MPA	9.75	11	BWO	8.12	11	GAO	9.44
12	SCA	9.56	12	GAO	10.62	12	GOA	11.00	12	MPA	10.38
13	MPA	9.75	13	SCA	12.50	13	MSA	12.62	13	SCA	10.62
Chi-square: 37.9273			*Chi-square: 64.8148*			*Chi-square: 42.0495*			*Chi-square: 51.2106*		
p-value: 0.000158			*p-value: 0.0000001*			*p-value: 0.000033*			*p-value: 0.000001*		

Table 4.8: Friedman Rankings using Random Forest

BestFit			**Mean Accuracy**			**WorstFit**			**Standard Deviation**		
Rank	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**	**Rank**	**Algo**	**Score**
1	MSA	3.75	1	BSLO	2.25	1	GOA	4.25	1	BSLO	2.69
2	BSLO	3.75	2	MSA	3.25	2	SSA	5.00	2	MSA	4.44
3	BWO	4.12	3	BWO	4.25	3	GAO	5.38	3	BWO	5.44
4	SSA	5.50	4	DO	5.44	4	DO	5.62	4	WOA	5.88
5	DO	5.88	5	MSO	6.50	5	SCA	5.62	5	MSO	6.12
6	PEOA	7.31	6	PEOA	6.75	6	MPA	6.75	6	DO	7.12
7	SCA	7.44	7	SSA	7.50	7	PEOA	7.12	7	SCA	7.62
8	GOA	7.75	8	SABO	7.62	8	SABO	7.12	8	SSA	7.62
9	SABO	7.88	9	WOA	8.19	9	BWO	7.12	9	SABO	8.06
10	MSO	8.81	10	SCA	8.62	10	WOA	8.38	10	PEOA	8.75
11	WOA	9.12	11	GOA	9.75	11	MSO	8.88	11	MPA	8.81
12	MPA	9.56	12	MPA	10.00	12	MSA	9.38	12	GOA	9.00
13	GAO	10.12	13	GAO	10.88	13	BSLO	10.38	13	GAO	9.44
Chi-square: 31.1967			*Chi-square: 43.9066*			*Chi-square: 21.3791*			*Chi-square: 25.3361*		
p-value: 0.001838			*p-value: 0.000016*			*p-value: 0.045097*			*p-value: 0.013308*		

4.3 Classifier-Wise Evaluation of Feature Selection Performance

The wrapper-based feature selection framework was used to evaluate the hybrid and individual algorithms across seven real-world credit scoring datasets using three classifiers: Logistic Regression, Naïve Bayes, and Random Forest. Each algorithm's performance was assessed based on four evaluation metrics—Mean Accuracy (average classification accuracy over 25 runs), Best Accuracy (maximum accuracy achieved), Worst Accuracy (minimum accuracy indicating reliability), and Standard Deviation (variance across runs indicating consistency). These metrics were chosen to mirror those used in the optimization phase while being tailored to classification performance.

The comparative tables (4.9, 4.10, 4.11, 4.12) illustrate the dominance of the MSO-GOA hybrid across all evaluation metrics and classifiers. In most cases, the hybrid achieved the highest overall mean accuracy and the lowest standard deviation, affirming its ability to provide compact yet high-performing feature subsets consistently.

Table 4.9: Mean Accuracy Comparative Tables across Classifiers

(a) Logistic Regression

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	89.86%	88.70%	84.64%	87.10%	78.33%	75.94%	88.99%	88.19%
Japan	79.47%	74.21%	75.19%	78.65%	61.58%	71.58%	78.50%	79.62%
crx	78.35%	79.70%	77.97%	78.80%	60.15%	64.14%	79.48%	78.80%
german1	80.40%	80.30%	78.10%	78.90%	73.80%	72.50%	78.15%	77.55%
hmeq	78.41%	77.84%	77.17%	77.74%	77.35%	77.13%	77.50%	78.48%
Taiwan	80.41%	78.15%	77.80%	77.81%	77.87%	77.68%	77.90%	78.37%
Loan Data	73.47%	72.98%	74.00%	72.98%	74.29%	75.47%	73.10%	74.00%
Overall Mean	**80.05%**	78.84%	77.84%	78.85%	71.90%	73.49%	79.09%	79.29%

(b) Naïve Bayes

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	89.54%	87.68%	85.29%	86.45%	87.46%	75.29%	86.83%	86.76%
Japan	77.89%	78.50%	76.39%	79.40%	78.12%	71.13%	78.77%	80.45%
crx	79.29%	79.02%	78.87%	79.30%	79.63%	66.69%	79.25%	80.00%
german1	78.70%	73.80%	75.15%	78.15%	77.00%	69.00%	77.65%	74.35%
hmeq	77.87%	76.24%	75.61%	77.40%	77.31%	75.91%	76.70%	77.65%
Taiwan	80.86%	80.16%	76.59%	81.01%	79.59%	78.18%	81.59%	81.61%
Loan Data	72.20%	73.27%	74.08%	74.53%	74.78%	66.00%	74.25%	74.82%
Overall Mean	**79.48%**	78.38%	77.43%	79.46%	79.13%	71.74%	79.29%	79.38%

(c) Random Forest

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	89.86%	81.09%	87.68%	87.75%	79.71%	81.74%	87.90%	88.12%
Japan	78.57%	66.84%	71.88%	71.35%	71.96%	72.03%	72.93%	72.86%
crx	78.05%	70.45%	70.15%	71.96%	74.36%	71.05%	71.58%	74.94%
german1	80.60%	73.30%	78.25%	74.95%	72.70%	70.90%	76.25%	76.80%
hmeq	78.60%	76.58%	78.93%	77.93%	74.36%	74.66%	78.79%	79.39%
Taiwan	81.64%	80.84%	81.78%	81.50%	80.97%	80.51%	81.48%	81.97%
Loan Data	73.92%	73.55%	74.73%	74.00%	71.27%	70.86%	74.41%	74.00%
Overall Mean	**80.18%**	74.66%	77.63%	77.06%	75.05%	74.54%	77.62%	78.30%

Table 4.10: Best Fit Accuracy Comparative Tables across Classifiers

(a) Logistic Regression

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	93.48%	92.75%	89.13%	92.75%	86.96%	86.23%	91.30%	91.30%
Japan	86.47%	81.95%	82.71%	82.46%	78.95%	87.22%	81.95%	83.21%
crx	86.47%	84.90%	82.71%	83.46%	78.20%	78.20%	87.22%	83.46%
german1	83.00%	84.00%	81.50%	85.50%	78.50%	78.00%	82.00%	81.50%
hmeq	80.06%	81.70%	79.61%	79.17%	78.72%	79.02%	79.46%	80.65%
Taiwan	82.27%	78.73%	78.15%	78.62%	78.70%	78.68%	78.63%	81.15%
Loan Data	76.73%	78.37%	78.37%	76.33%	75.92%	79.59%	76.33%	76.73%
Overall Mean	**84.07%**	83.20%	81.74%	82.61%	79.42%	80.99%	82.41%	82.57%

(b) Naïve Bayes

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	92.03%	90.58%	92.75%	90.58%	92.03%	88.41%	90.58%	92.75%
Japan	84.96%	83.46%	84.96%	81.95%	81.95%	81.20%	83.46%	84.96%
crx	84.21%	81.20%	82.71%	82.71%	85.71%	78.20%	84.96%	83.46%
german1	81.50%	78.00%	78.50%	83.00%	81.00%	74.50%	83.00%	78.50%
hmeq	79.46%	78.72%	78.42%	79.61%	79.91%	78.87%	78.72%	79.61%
Taiwan	82.95%	81.42%	81.32%	82.18%	80.73%	80.85%	82.38%	82.63%
Loan Data	83.55%	76.33%	77.55%	79.18%	78.37%	73.47%	78.37%	79.59%
Overall Mean	**84.09%**	81.39%	82.32%	82.74%	82.81%	79.36%	83.07%	83.07%

(c) Random Forest

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	94.20%	90.58%	91.30%	92.75%	89.86%	93.48%	89.86%	92.03%
Japan	85.71%	78.20%	79.70%	79.70%	79.70%	78.20%	80.45%	79.20%
crx	84.21%	80.45%	78.95%	81.95%	80.45%	78.95%	79.70%	79.95%
german1	84.00%	79.00%	83.00%	78.00%	78.50%	75.00%	78.50%	81.00%
hmeq	81.25%	79.91%	80.95%	80.51%	77.53%	76.49%	80.06%	81.10%
Taiwan	81.95%	81.53%	82.47%	81.77%	82.27%	82.05%	81.97%	82.85%
Loan Data	77.55%	76.73%	78.37%	77.96%	73.88%	76.33%	79.18%	77.96%
Overall Mean	**84.12%**	80.91%	82.11%	81.81%	80.31%	80.07%	81.39%	82.01%

Table 4.11: Worst Fit Accuracy Comparative Tables across Classifiers

(a) Logistic Regression

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	86.23%	86.23%	74.64%	81.16%	68.12%	65.22%	84.03%	80.61%
Japan	75.19%	53.38%	65.41%	72.93%	54.89%	53.38%	72.94%	73.94%
crx	60.15%	75.19%	66.92%	73.68%	53.38%	51.88%	73.19%	72.19%
german1	79.00%	77.50%	75.00%	72.50%	67.50%	65.50%	72.50%	75.00%
hmeq	76.04%	74.55%	75.15%	75.45%	76.04%	74.40%	75.15%	73.45%
Taiwan	77.63%	77.43%	77.08%	76.52%	77.03%	76.45%	77.18%	77.10%
Loan Data	69.80%	69.39%	72.24%	68.98%	70.20%	71.43%	68.98%	71.43%
Overall Mean	**74.86%**	73.38%	72.35%	74.46%	66.74%	65.47%	74.85%	74.82%

(b) Naïve Bayes

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	86.09%	84.06%	77.54%	83.06%	80.43%	64.49%	84.51%	84.51%
Japan	74.66%	73.68%	68.42%	75.69%	72.93%	55.64%	74.94%	77.44%
crx	76.65%	76.69%	73.68%	76.44%	75.19%	48.87%	74.44%	75.94%
german1	76.50%	68.00%	71.50%	74.00%	74.00%	60.50%	74.50%	70.50%
hmeq	75.30%	74.26%	72.17%	74.15%	74.85%	72.32%	74.15%	75.30%
Taiwan	75.20%	79.02%	60.63%	79.62%	78.08%	75.23%	80.65%	80.00%
Loan Data	69.39%	66.53%	69.39%	70.43%	71.84%	51.02%	69.80%	69.80%
Overall Mean	**76.26%**	74.61%	70.48%	76.20%	75.33%	61.15%	76.14%	76.21%

(c) Random Forest

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	84.06%	66.67%	84.06%	82.61%	63.77%	73.19%	85.51%	86.23%
Japan	73.68%	51.88%	63.91%	60.90%	63.16%	65.41%	60.15%	60.90%
crx	69.92%	59.40%	62.41%	63.91%	69.17%	62.41%	60.15%	63.16%
german1	78.00%	66.00%	72.50%	71.00%	66.50%	66.50%	73.00%	72.00%
hmeq	75.89%	74.55%	76.34%	75.30%	64.58%	72.32%	76.34%	74.40%
Taiwan	81.20%	78.87%	81.33%	81.32%	79.38%	78.72%	80.47%	81.28%
Loan Data	69.39%	70.20%	71.43%	71.02%	66.94%	67.35%	72.24%	66.12%
Overall Mean	**76.02%**	66.80%	73.14%	72.29%	67.64%	69.41%	72.55%	72.01%

Table 4.12: Standard Deviation Comparative Tables across Classifiers

(a) Logistic Regression

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	1.99	2.44	4.26	3.40	6.15	6.62	1.85	2.62
Japan	3.19	10.30	4.95	3.77	7.34	11.98	2.10	2.73
crx	3.14	2.70	4.78	3.68	6.76	10.24	3.97	2.62
german1	1.22	1.97	2.26	3.80	3.28	4.08	2.98	2.55
hmeq	1.21	2.09	1.47	1.40	0.93	1.59	1.39	1.58
Taiwan	1.78	0.36	0.35	0.64	0.49	0.60	0.44	1.16
Loan Data	2.03	2.88	2.11	2.19	1.70	2.65	2.30	1.77
Overall Mean	**2.08**	3.25	2.88	2.70	3.81	5.39	2.15	2.15

(b) Naïve Bayes

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	1.28	2.27	5.22	2.37	3.30	7.03	1.65	1.98
Japan	2.61	2.91	5.32	1.91	2.76	8.59	2.96	2.42
crx	1.38	1.52	3.16	1.73	3.20	7.66	3.31	2.08
german1	1.60	2.93	2.39	2.45	2.25	4.46	2.44	2.20
hmeq	1.31	1.40	1.96	1.89	1.57	2.02	1.24	1.19
Taiwan	2.46	0.77	7.30	0.80	0.79	1.55	0.55	0.70
Loan Data	2.77	2.76	3.27	2.51	2.11	7.79	2.75	3.02
Overall Mean	**1.92**	2.08	4.09	1.95	2.28	5.59	2.13	1.94

(c) Random Forest

Dataset	**Hybrid**	**GOA**	**MSO**	**DOA**	**BWO**	**MSA**	**SSA**	**WOA**
australian	2.90	8.30	2.56	2.83	8.66	7.03	1.41	1.61
Japan	3.82	9.23	5.08	5.82	4.78	5.20	6.28	6.64
crx	4.60	7.19	6.84	5.55	3.79	7.14	6.53	5.23
german1	1.79	4.37	3.01	1.89	4.49	2.80	2.32	3.12
hmeq	1.60	1.55	1.53	1.73	3.66	1.52	1.14	1.97
Taiwan	0.23	0.91	0.36	0.17	1.17	1.26	0.47	0.44
Loan Data	2.66	2.36	2.24	2.27	1.92	3.48	2.00	3.39
Overall Mean	**2.51**	4.84	3.09	2.89	4.07	4.06	2.88	3.20

4.4 Effectiveness of the Hybrid MSO-GOA Algorithm

The proposed MSO-GOA hybrid integrates the superior global exploration ability of MSO and the fine-tuned local exploitation of GOA. This balanced search behavior promotes faster convergence, better avoidance of local optima, and stable results.

In the feature selection phase, MSO-GOA consistently produced compact subsets with fewer features while achieving equal or superior classification accuracy compared to its components and competing methods. This efficiency, combined with its robust accuracy across classifiers and datasets, highlights its practical utility for interpretable and high-performing credit scoring models.

4.5 Visual Analysis of Comparative Results

To supplement numerical results, Figure 4.1 offers a visual comparison of MSO-GOA, MSO, and GOA across classifiers and metrics. The hybrid consistently outperforms its components, confirming that the designed synergy yields tangible performance gains.

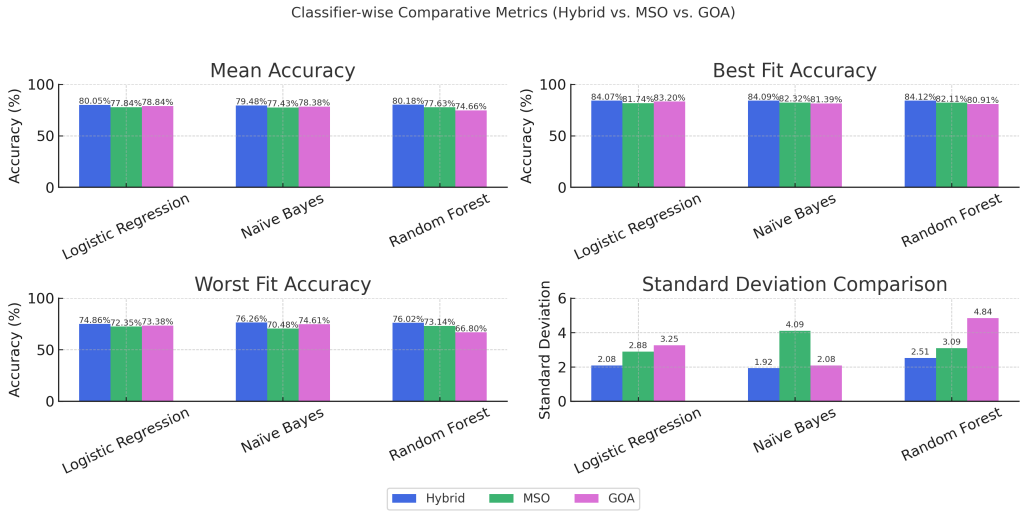

Figure 4.1: Classifier-wise Comparative Metrics (Hybrid vs. MSO vs. GOA)

4.6 Summary

The discussion confirms that the MSO-GOA hybrid is a robust and versatile optimization tool. Although it was not applied to standard benchmark functions, its component algorithms (MSO and GOA) showed strong performance in those settings, which justifies their selection. The hybrid itself excelled in producing compact and reliable feature subsets across seven real-world credit scoring datasets and three classifiers. Its consistent superiority across four metrics and validated through statistical tests makes it a strong candidate for practical deployment in financial modeling and decision intelligence.

Chapter 5

Conclusion

In this research, we addressed the important challenge of feature selection for credit scoring by developing and evaluating a novel hybrid metaheuristic algorithm. We began by motivating the need for feature selection in high-dimensional datasets, especially in finance division where model interpretability and accuracy are critical. Our research objective was to compare and improve the performance of existing metaheuristic optimizers through hybridization.

We implemented a structured five-phase methodology. Phase 1 involved benchmarking thirteen metaheuristic algorithms on a suite of 76 classical optimization functions (unimodal, multimodal, composite). This identified the strongest performers in terms of convergence and consistency. Phases 2 and 3 converted each algorithm into a binary feature selector (using a logistic sigmoid transfer function) and applied them in a wrapper framework on seven public credit scoring datasets, with Logistic Regression as the first classifier. Phase 4 repeated this with Naïve Bayes and Random Forest to test

generalization. In all cases, we recorded accuracy, subset size, and statistical summaries over multiple runs. Finally, Phase 5 used the top algorithms from these experiments to construct a hybrid optimizer, which we tested in the same feature-selection context.

The key findings are as follows. The proposed hybrid MSO-GOA algorithm was evaluated on the same seven credit scoring datasets using the exact same three classifiers—Logistic Regression, Naïve Bayes, and Random Forest—with identical experimental parameters as the individual algorithms (i.e., number of runs, number of agents, and population size). Unlike the individual metaheuristics that were statistically compared using the Friedman test, MSO-GOA's performance was assessed based on the overall average of four metrics—BestFit Accuracy, Mean Accuracy, WorstFit Accuracy, and Standard Deviation—computed across all datasets. The hybrid consistently achieved higher average Mean Accuracy and BestFit Accuracy while maintaining lower WorstFit and Standard Deviation values compared to most individual algorithms, indicating a strong balance between accuracy, robustness, and model simplicity.

Furthermore, MSO-GOA typically selected significantly fewer features while maintaining or improving classification accuracy relative to the best-performing individual algorithms. For instance, with Logistic Regression, the hybrid often reduced the feature set by approximately 20–30% without any noticeable loss in predictive performance. This feature reduction not only simplifies the model but also enhances interpretability and generalization. Although MSO-GOA was not included in the Friedman rankings (Tables 4.6–4.8), its superior average performance across all classifiers and datasets highlights its practical effectiveness. The selected features frequently

included domain-relevant variables such as debt-to-income ratio and credit history length, suggesting that the hybrid effectively identified meaningful patterns in the data.

Taken together, these results imply that a well-designed hybrid metaheuristic can outperform its component algorithms in both abstract optimization and applied feature selection. MSO-GOA was able to balance fast convergence with broad search, yielding credit scoring models that are both accurate and efficient (smaller and simpler). In practical terms, this means financial institutions could use such a hybrid method to build credit models that generalize better and are easier to interpret.

Implications of the results: In credit scoring, reducing the number of input features has immediate benefits. A model with fewer features is simpler to audit and explain, which is important for regulatory compliance (e.g. Basel III and GDPR stress transparency). Leaner models also reduce overfitting and improve stability when new data arrive. The fact that MSO-GOA maintained high accuracy with fewer features suggests it isolated the core predictors of default risk. From a broader machine-learning perspective, our findings demonstrate that hybrid metaheuristics can be a powerful tool for feature selection in any domain where high dimensionality is an issue.

Limitations: This study has several limitations. We evaluated performance on seven standard credit datasets (ranging from hundreds to tens of thousands of records), which cover diverse but limited cases. Real-world portfolios might have millions of customers or more complex co-variate structures, so scalability needs further testing. We also framed feature selection as a single-objective problem (maximize accuracy), with feature reduction as a byproduct. In practice, there is an inherent trade-off between accuracy and

model complexity. An explicit multi-objective treatment (e.g. Pareto optimization of accuracy vs. number of features) might reveal different insights about this trade-off.

Additionally, our algorithms were run with fixed parameters and budget; exploring the sensitivity of results to these settings is necessary. The binarization (sigmoid) approach is standard but not guaranteed optimal; alternative transfer functions or thresholding could change performance. Finally, metaheuristic optimization has stochastic variability. Although we averaged results over multiple runs and used statistical tests, any single run may perform differently.

Future work: Several avenues can extend this research. First, implementing a multi-objective feature selection framework would allow users to balance accuracy against sparsity explicitly. For example, a Pareto-based algorithm (like NSGA-II or MOEA/D) based on the MSO-GOA framework could yield a front of high-accuracy, low-feature solutions. Second, we could incorporate domain knowledge into the search, such as penalizing features correlated with protected attributes (to enhance fairness) or preferring readily available features (to improve practicality). Third, the algorithmic approach itself can be expanded: other metaheuristics can be hybridized, or the hybrid can adaptively switch strategies during a run. Exploring different discretizations methods and automated parameter tuning could further improve results.

From a deployment perspective, improving scalability is critical. Parallel or distributed versions of MSO-GOA could handle much larger datasets or allow more thorough searches. Investigating online or incremental feature selection methods would enable the model to update selected features as new

data streams in. Lastly, since machine learning in credit scoring is evolving, testing the selected features and hybrid algorithm with more complex models (e.g. gradient boosting, neural networks) and in other domains (medical diagnosis, fraud detection, etc.) would validate the generality of our approach.

In summary, this work has shown that a thoughtfully constructed hybrid metaheuristic can substantially improve feature selection in credit scoring applications, producing simpler models without sacrificing accuracy. The MSO-GOA algorithm and the comprehensive experimental framework developed here provide a solid foundation for further research. The insights gained—about algorithmic balance, feature reduction, and rigorous comparison—will guide future enhancements in feature selection methods and their deployment in real-world predictive modeling.

References

[1] J. Barrera-García, F. Cisternas-Caneo, B. Crawford, M. Gómez Sánchez, and R. Soto, "Feature selection problem and metaheuristics: A systematic literature review about its formulation, evaluation and applications," *Biomimetics*, vol. 9, no. 1, pp. 1–48, 2024. doi: 10.3390/biomimetics9010009

[2] O. M. Alyasiri, Y.-N. Cheah, A. K. Abasi, and O. M. Al-Janabi, "Wrapper and hybrid feature selection methods using metaheuristic algorithms for English text classification: A systematic review," *IEEE Access*, vol. 10, pp. 39833–39853, Apr. 2022. doi: 10.1109/ACCESS.2022.3165814

[3] S. Kaur, Y. Kumar, A. Koul, and S. K. Kamboj, "A systematic review on metaheuristic optimization techniques for feature selections in disease diagnosis: Open issues and challenges," *Archives of Computational Methods in Engineering*, vol. 30, pp. 1863–1895, 2023. doi: 10.1007/s11831-022-09853-1

[4] P. Agrawal, H. F. Abutarboush, T. Ganesh, and A. W. Mohamed, "Metaheuristic algorithms on feature selection: A survey of one decade of research (2009–2019)," *IEEE Access*, vol. 9, pp. 26766–26791, 2021. doi: 10.1109/ACCESS.2021.3056407

[5] T. Dokeroglu, A. Deniz, and H. E. Kiziloz, "A comprehensive survey on recent metaheuristics for feature selection," *Neurocomputing*, vol. 535, pp. 126354, 2023. doi: 10.1016/j.neucom.2022.04.083

[6] M. Nssibi, G. Manita, and O. Korbaa, "Advances in nature-inspired metaheuristic optimization for feature selection problem: A comprehensive

survey," *Computer Science Review*, vol. 49, 100540, 2023. doi: 10.1016/j.cosrev.2023.100559

[7] O. O. Akinola, A. E. Ezugwu, J. O. Agushakaand, R. A. Zitar, and L. Abualigah, "Multiclass feature selection with metaheuristic optimization algorithms: A review," *Neural Computing and Applications*, vol. 34, pp. 9613–9642, 2022. doi: 10.1007/s00521-022-07705-4

[8] R. Arun Kumar, J. Vijay Franklin, and N. Koppula, "A comprehensive survey on metaheuristic algorithm for feature selection techniques," *Materials Today: Proceedings*, vol. 62, no. 5, pp. 3196–3202, 2022. doi: 10.1016/j.matpr.2022.04.803

[9] D. Wang, H. Chen, T. Li, J. Wan, and Y. Huang, "A novel quantum grasshopper optimization algorithm for feature selection," *International Journal of Approximate Reasoning*, vol. 124, pp. 194–211, 2020. doi: 10.1016/j.ijar.2020.08.010

[10] P. D. Sheth, S. T. Patil, and M. L. Dhore, "Evolutionary computing for clinical dataset classification using a novel feature selection algorithm," *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 5, pp. 2064–2075, 2022. doi: 10.1016/j.jksuci.2020.12.012

[11] J. He, S. Zhao, J. Ding, and Y. Wang, "Mirage search optimization: Application to path planning and engineering design problems," *Advances in Engineering Software*, vol. 183, p. 103883, 2025. doi: 10.1016/j.advengsoft.2025.103883

[12] P. Trojovský, "A new human-based metaheuristic algorithm for solving

optimization problems based on preschool education," *Scientific Reports*, vol. 13, no. 1, p. 21472, Dec. 2023. doi: 10.1038/s41598-023-48462-1

[13] Z. Montazeri, T. Niknam, J. Aghaei, O. P. Malik, M. Dehghani, and G. Dhiman, "Golf Optimization Algorithm: A new game-based metaheuristic algorithm and its application to energy commitment problem considering resilience," *Biomimetics*, vol. 8, no. 5, Art. no. 386, 2023. doi: 10.3390/biomimetics8050386

[14] J. Bai, H. Nguyen-Xuan, E. Atroshchenko, G. Kosec, L. Wang, and M. Abdel Wahab, "Blood-sucking leech optimization (BSLO): A novel nature-inspired algorithm for optimization problems," *Advances in Engineering Software*, vol. 195, Art. no. 103696, 2024. doi:10.1016/j.advengsoft.2024.103696

[15] O. Alsayyed, T. Hamadneh, H. Al-Tarawneh, M. Alqudah, S. Gochhait, I. Leonova, O. P. Malik, and M. Dehghani, "Giant Armadillo Optimization: A New Bio-Inspired Metaheuristic Algorithm for Solving Optimization Problems," *Biomimetics*, vol. 8, no. 8, p. 619, 2023. doi: 10.3390/biomimetics8080619

[16] P. Trojovský and M. Dehghani, "Subtraction-Average-Based Optimizer: A New Swarm-Inspired Metaheuristic Algorithm for Solving Optimization Problems," *Biomimetics*, vol. 8, no. 2, p. 149, 2023. doi: 10.3390/biomimetics8020149

[17] C. Zhong, G. Li, and Z. Meng, "Beluga Whale Optimization: A Novel Nature-Inspired Metaheuristic Algorithm," *Knowledge-Based Systems*, vol. 250, p. 109215, 2022. doi: 10.1016/j.knosys.2022.109215

[18] S. Zhao, T. Zhang, S. Ma, and M. Chen, "Dandelion optimizer: A nature-inspired metaheuristic algorithm," *Engineering Applications of Artificial Intelligence*, vol. 114, p. 105082, 2022. doi: 10.1016/j.engappai.2022.105075

[19] A. Faramarzi, M. Heidarinejad, S. Mirjalili, and A. H. Gandomi, "Marine Predators Algorithm: A Nature-Inspired Metaheuristic," *Expert Systems with Applications*, vol. 152, p. 113377, 2020. doi: 10.1016/j.eswa.2020.113377

[20] S. Mirjalili and A. Lewis, "The Whale Optimization Algorithm," *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016. doi:10.1016/j.advengsoft.2016.01.008

[21] S. Mirjalili, "SCA: A sine cosine algorithm for solving optimization problems," *Knowledge-Based Systems*, vol. 96, pp. 120–133, Mar. 2016. doi: 10.1016/j.knosys.2015.12.022

[22] A. A. Mohamed, Y. S. Mohamed, A. A. M. El-Gaafary, and A. M. Hemeida, "Optimal power flow using moth swarm algorithm," *Electric Power Systems Research*, vol. 142, pp. 190–206, 2017. doi: 10.1016/j.epsr.2016.09.025

[23] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems," *Advances in Engineering Software*, vol. 114, pp. 163–191, 2017. doi: 10.1016/j.advengsoft.2017.07.002

[24] J. Laborda and S. Ryoo, "Feature selection in a credit scoring model,"

Mathematics, vol. 9, no. 7, pp. 1–22, 2021. doi: 10.3390/math9070746

[25] G. Nie, W. Rowe, L. Zhang, Y. Tian, and Y. Shi, "Credit card churn forecasting by logistic regression and decision tree," *Expert Systems with Applications*, vol. 38, no. 12, pp. 15273–15285, 2011. doi: 10.1016/j.eswa.2011.05.027

[26] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, Feb. 1996. doi: 10.1109/3477.484436

[27] C. Charin, D. Ishak, M. A. A. M. Zainuri, B. Ismail, and M. K. M. Jamil, "A hybrid of bio-inspired algorithm based on Levy flight and particle swarm optimizations for photovoltaic system under partial shading conditions," vol. 221, pp. 116–132, 2021. doi: 10.1016/j.solener.2021.01.049